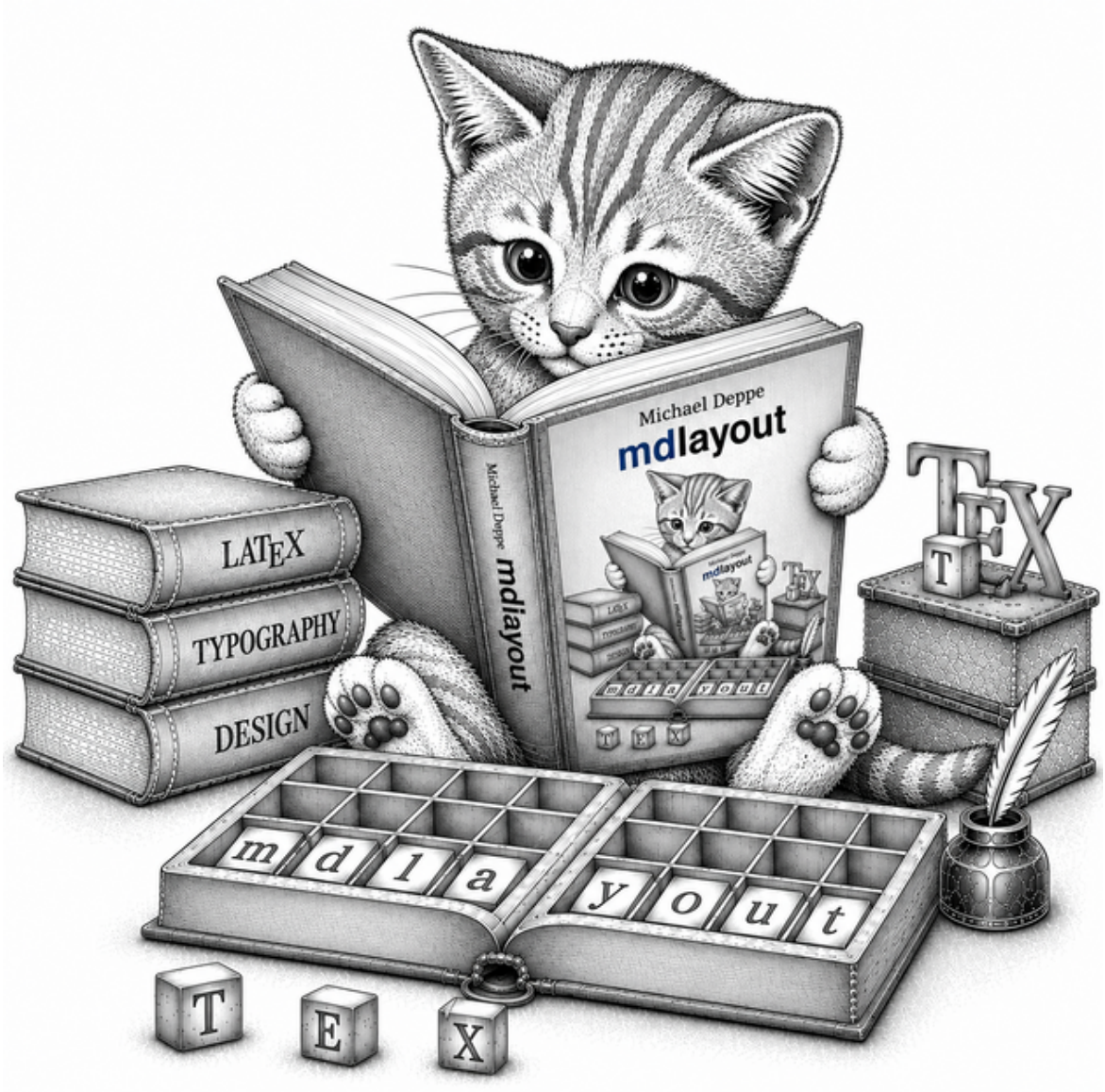
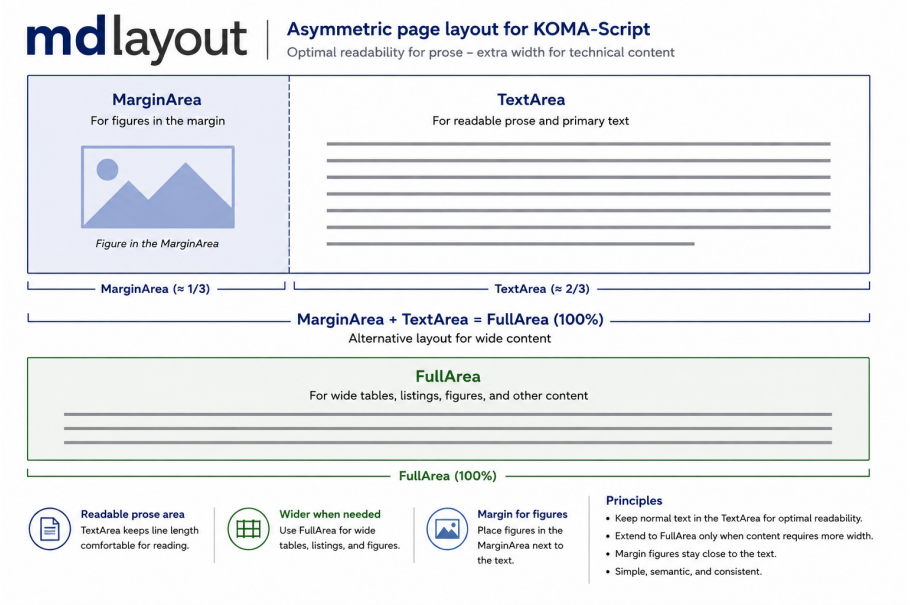
Michael Deppe

mdlayout



mdlayout

Asymmetric Page Layout
for KOMA-Script Technical Documentation



MarginArea + TextArea = FullArea

User's Guide

Version 0.16.5

Michael Deppe

2026/08/18

*Readable prose and wide technical material place conflicting demands on page layout. **md**layout resolves this conflict by introducing semantically named layout areas that provide additional width only where it is actually required.*

Contents

List of Figures

List of Tables

Colophon

This manual is itself an example of a technical document produced with **md**layout. The complete document source, including the accompanying `mdpreamble.tex`, is distributed as part of the package and may serve as a starting point for your own documentation projects.

Typeset with LuaLaTeX, KOMA-Script and **md**layout Version 0.16.5.
Compiled on 2026/09/04 at 10:12:00.

Preface

mdlayout was not created to provide another collection of layout macros. Its purpose is different. Like LaTeX itself, **md**layout encourages authors to describe the semantic meaning of document elements rather than their visual appearance. Instead of specifying how something should look, the author describes what it represents. A compiler listing is not simply verbatim text. A system log is not just another listing. A machine printout is fundamentally different from source code. Each of these objects has its own logical structure and, consequently, its own natural layout.

The environments provided by **md**layout therefore represent document types rather than formatting instructions. The package chooses an appropriate visual representation while preserving the logical properties of the underlying material.

This philosophy is perhaps most evident in the `Printout` environment. Rather than selecting a font size, the author specifies the logical width of the output, for example using `fitcolumns=132`. **md**layout automatically determines the largest suitable monospaced font that fits the available print area. Different appearances, such as plain output, green-bar paper or an IBM 1403 listing, are simply different *forms* of the same document type.

The same principle applies to the `ReferenceTable` environment. Instead of constructing a table from low-level column specifications, the author describes the role of its columns. A column may contain ordinary text, typewriter text or verbatim material, while other columns can automatically use the remaining available width. The table can therefore adapt to different layout areas without changing its semantic description. This is particularly useful in technical documentation, where tables frequently combine commands, literal syntax, default values and explanatory text. `ReferenceTable` treats these as properties of the information being presented rather than as instructions for constructing a particular LaTeX table.

Many ideas implemented in **md**layout originated from documenting real-world software projects, where source code, terminal sessions, system logs and generated output appear side by side. The package is therefore driven by practical requirements rather than visual effects. If **md**layout encourages authors to think first about the meaning of a document element before deciding how it should look, then it has achieved its primary objective.

Michael Deppe
August 2026

Chapter 1

Introduction

1.1 Why mdlayout?

LaTeX has become the de facto standard for scientific and technical publishing. Together with KOMA-Script it provides an excellent typographic foundation for books, reports, manuals, and reference documentation. Technical documentation, however, places demands on page layout that differ significantly from those of ordinary prose.

A typical technical manual contains program listings, terminal sessions, directory listings, configuration files, command syntax, flow diagrams, screenshots, wide tables, and reference material. Many of these objects require considerably more horizontal space than continuous text.

Increasing the width of every page appears to solve this problem. Unfortunately, it also produces long text lines that are more difficult to read and visually less balanced. The challenge is therefore straightforward:

> How can a document provide *additional width* only where it is actually needed while preserving *comfortable line lengths* for ordinary text?

mdlayout answers this question by separating the concepts of *readability* and *available page width*. Instead of treating every page as a single rectangular text block, the package divides the page into semantically named layout areas. Each area serves a specific purpose and can be selected directly by the author without calculating dimensions or modifying page geometry. The guiding principle is deliberately simple:

> **Use the smallest layout area that presents the content correctly.**

Ordinary prose remains inside the **TextArea**. Small illustrations or little symbols (see above) inside the **MarginArea** may accompany the text. Only content that genuinely requires additional width extends into the **HalfArea** or the **FullArea**. The result is a page layout that combines comfortable reading with the ability to present complex technical material without compromise.

1.2 Design Philosophy

The design of **md**layout follows a small number of simple principles.

1.2.1 Semantic instead of geometric layout

Authors should not think in terms of millimetres or page dimensions. Instead of asking

> “How much wider should this table become?”

the author asks

> “Which layout area best represents this content?”

This semantic approach keeps the document source readable while producing a consistent layout throughout the entire document.

1.2.2 Simplicity

Every public command has a clearly defined purpose. Whenever several implementation strategies are possible, **md**layout deliberately favours the solution that keeps the document source understandable, predictable, and easy to maintain.

1.2.3 Explicit author control

The package intentionally avoids unnecessary automation. For example, the `MarginFigure` environment is not a floating environment. A margin figure appears exactly where the author places it in the source document. Automatic repositioning would transform it into another floating mechanism and would therefore contradict its intended purpose. Whenever automatic placement is desired, the standard floating `figure` environment remains the appropriate choice.

1.2.4 Built on KOMA-Script

mdlayout is not a document class. Instead, it extends KOMA-Script with a compact collection of layout tools specifically designed for technical documentation while preserving the typographic quality and familiar workflow provided by KOMA-Script.

1.3 The Layout Model

The entire package is based on three fundamental layout areas (Figure 1.1).

- The **TextArea** contains ordinary prose.
- The **MarginArea** accompanies the TextArea and is primarily intended to keep
  - the TextArea narrow,
  - to provide space for section titles, and
  - for small non-floating illustrations placed using the
  - **MarginFigure** environment.

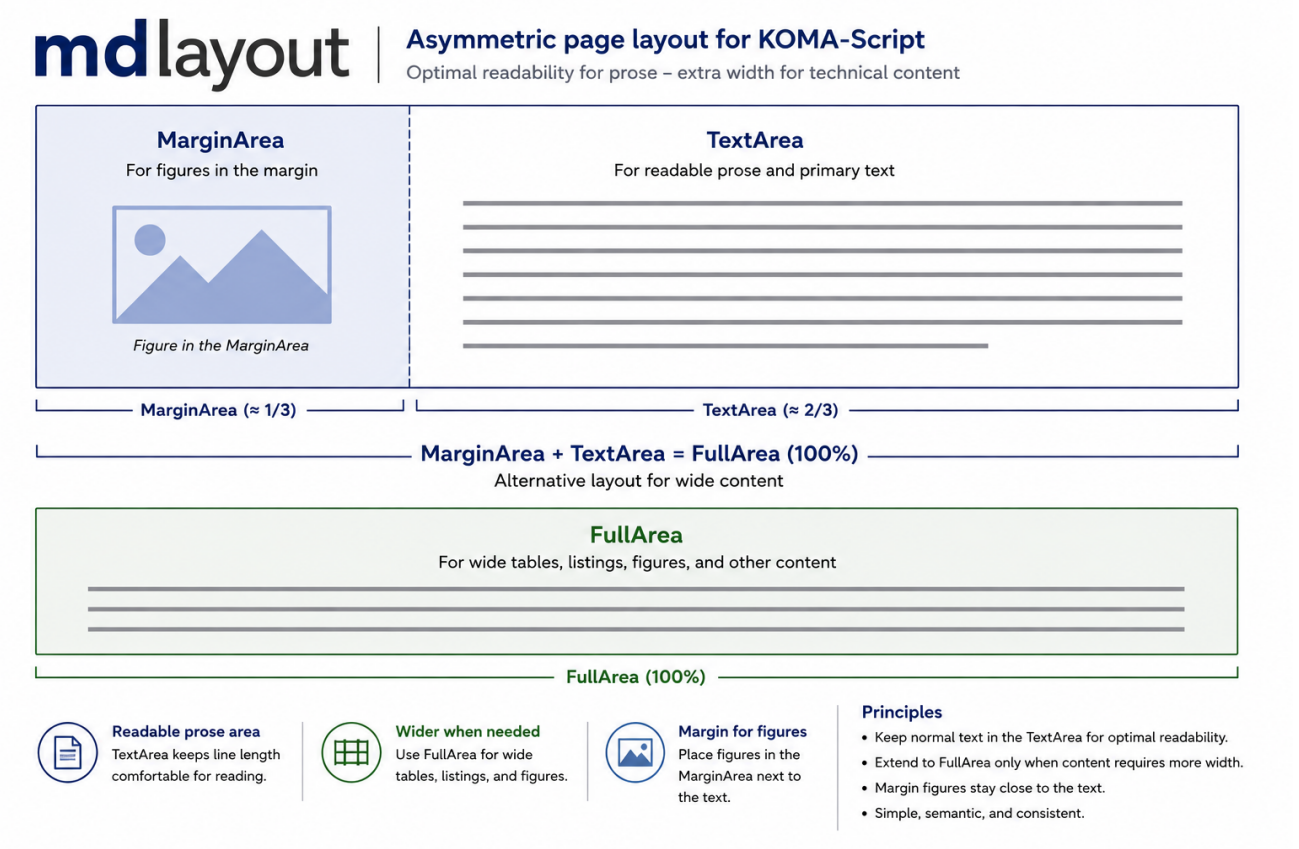


Figure 1.1: Asymmetric Layout of **md**layout. This figure uses the space provided by **FullArea**

- Together both, the **MarginArea** and the **TextArea** area form the **FullArea**, which becomes available whenever technical content requires the complete usable page width.
- Every other environment provided by mdlayout is derived from this simple model.

1.4 Scope

mdlayout concentrates on practical requirements commonly encountered in technical books and reference manuals. It provides dedicated layout environments for

- readable continuous prose,
- document directories,
- wide tables,
- wide figures,
- long tables,
- terminal output and listings,
- full-width chapter headings,
- non-floating margin figures,
- and long or wide printouts, e.g. logfiles.

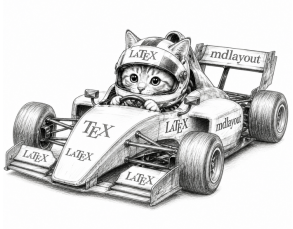
The package intentionally avoids decorative features and unnecessary complexity. Its objective is not to replace KOMA-Script but to complement it with a small, consistent, and semantically oriented layout system for technical documentation.

1.5 Meet Tixi

Throughout this documentation you will occasionally encounter *Tixi*, the little **md**layout cat. Tixi has developed a particular interest in TEX, LATEX, and asymmetric page layouts. She measures margins, moves type around, experiments with boxes, and occasionally operates machinery that probably should not be entrusted to a kitten.
Tixi prefers semantic layout instructions to mysterious negative horizontal spaces and therefore feels quite at home in **md**layout. Her illustrations accompany examples of **md**layout concepts and provide a visual counterpart to some of the more technical parts of this documentation.

Chapter 2

Quick Start



This chapter demonstrates the most frequently used features of **md**layout. The examples are intentionally short and can be copied directly into your own documents. In most technical books only four concepts are required:

- ordinary text remains in the **TextArea**,
- slightly wider material uses the **HalfArea**,
- large technical objects use the **FullArea**,
- small accompanying illustrations are placed using **MarginFigure**.

No manual page geometry calculations are necessary.

2.1 Ordinary Text

No special environment is required for continuous prose. Simply write

```
This is an ordinary paragraph.

It is automatically typeset inside the TextArea.
```

Continuous text should normally remain inside the TextArea because this provides the most comfortable line length for reading.

2.2 Using HalfArea

Whenever a little more horizontal space is required, the text can be extended into the HalfArea.

```
\begin{HalfArea}

 This paragraph is wider than the normal TextArea but does not occupy
 the complete FullArea.

\end{HalfArea}
```

Result of the last command using `HalfArea`:

This paragraph is wider than the normal TextArea but does not occupy the complete FullArea.

Typical applications are:

- command descriptions,
- option lists,
- API documentation,
- configuration files.

2.3 Using FullArea

Large technical material should use the complete available page width.

```
\begin{FullArea}

 \includegraphics[width=\linewidth]{architecture.pdf}

\end{FullArea}
```

Typical applications include

- program listings,
- terminal sessions,
- screenshots,
- flow diagrams,
- wide tables,
- long command lines.

2.4 Margin Figures

Small illustrations that accompany the surrounding text may be placed inside the MarginArea (see Figure 2.1).

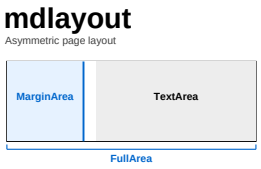


Figure 2.1: The basic **md**layout layout model

```
\begin{MarginFigure}

 \includegraphics[width=\mdMarginWidth]{mdlayout-margin-logo.pdf}
 \caption{The basic \mdlayout{} layout model}
 \label{fig:margin-logo}

\end{MarginFigure}
```

The `MarginFigure` environment places small figures next to the associated text, while the normal body text remains within the `TextArea`. Unlike the standard `figure` environment, `MarginFigure` is intentionally *non-floating*. It therefore appears exactly at the position where it is placed in the source document. As a consequence, the author must take some care when placing `MarginFigure` environments, since they may overlap with other material in the `MarginArea` if insufficient vertical space is available.

2.5 Wide Figures

Whenever automatic figure placement is desired, use `widefigure`.

```
\begin{widefigure}[htbp]

  \includegraphics[width=\linewidth]{system-overview.pdf}

  \caption{Overall system architecture}

\end{widefigure}
```

The environment behaves like a normal floating `figure` but automatically uses the FullArea.

2.6 Choosing the Correct Area

A simple rule is sufficient for almost every document.

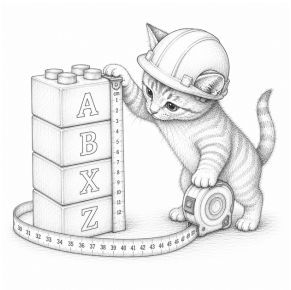
TextArea	Continuous prose.
HalfArea	Moderately wide technical material.
FullArea	Large technical objects requiring the maximum available width.
MarginFigure	Small illustrations accompanying the surrounding text.

Whenever in doubt, choose the smallest area that presents the content comfortably. This principle keeps documents visually balanced while making additional page width available exactly where it is needed.

2.7 Semantic Layout

One of the fundamental design goals of **md**layout is the separation of *what* should happen from *how* it is implemented. Traditional layout packages often require the author to think in terms of physical dimensions:

```
\begin{adjustwidth}{-37mm}{0mm}
 ...
\end{adjustwidth}
```



Such code describes the implementation rather than the intention. The value `-37mm` has no semantic meaning. A reader cannot determine why this particular value was chosen or what the author intended to achieve. In contrast, **md**layout encourages semantic document source:

```
\begin{FullArea}
 ...
\end{FullArea}
```

The source now describes the purpose rather than the geometry. The author no longer asks

> "*How many millimetres should I move this paragraph?*"

Instead, the question becomes

> "*Which layout area best represents this content?*"

This makes the document easier to understand, easier to maintain, and largely independent of the underlying page geometry. The same philosophy applies throughout the package:

Environment	**Meaning**
`TextArea`	Continuous prose
`HalfArea`	Moderately wide material
`FullArea`	Maximum available width
`DirectoryArea`	Document directories
`MarginFigure`	Small accompanying illustrations
`Printout`	Long logfiles and printout contents
`ReferenceTable`	Long tables for commands and options

The package therefore describes *what belongs where* rather than *how far something should be moved*.

Chapter 3

Installation and Requirements

3.1 LaTeX Engine

The **md**layout package requires LuaLaTeX and **UTF-8** encoding. When using an integrated LaTeX development environment that understands TeX editor directives (`magic comments`), such as **TeXstudio**, **TeXShop**, or **TeXworks**, the following two lines should be placed at the beginning of the document:

```
% !TEX TS-program = lualatex
% !TeX encoding = UTF-8
```

3.2 Requirements

The **md**layout package extends the page layout and heading mechanisms provided by KOMA-Script. It supports the following KOMA-Script document classes:

- `scrartcl`,
- `scrreprt`,
- `scrbook`.

Other document classes are not supported. The package loads the following dependencies:

- `kvoptions`,
- `etoolbox`,
- `changepage`,
- `scrlayer-scrpage`,
- `xcolor`,
- `graphicx`,
- `caption`.

The `fancyhdr` package is not supported. Page headers and footers are configured using `scrlayer-scrpage`.

3.3 Basic Package Loading

The package can be loaded without explicit options:

```
\usepackage{mdlayout}
```

The default configuration uses a margin area of `38mm` and a gutter of `5mm`. A typical explicit configuration (here with the default values) is:

```
\usepackage[
marginwidth=38mm,
gutter=5mm,
wideheadings=true,
wideheadfoot=true,
rightmargin=20mm,
headsepline=0.8pt,
plainheadsepline=false,
bottommargin=30mm,
chapterrule=2.0pt,
sectionrule=1.4pt,
headingrulecolor=black,
chapterrulegap=0.55\baselineskip,
sectionrulegap=0.35\baselineskip
```

```
]mdlayout
```

The normal KOMA-Script text width is retained as the width of the **TextArea**. The additional **MarginArea** and gutter extend to the left of this column.

3.4 Recommended Loading Order

The KOMA-Script document class must be selected before loading **md**layout:

```
\documentclass{scrbook}

\usepackage[
marginwidth=35mm,
gutter=5mm
]{mdlayout}

\input{mdpreamble}
```

Packages that configure fonts, bibliography, hyperlinks, tables, listings, or document-specific colours may be loaded separately in the document preamble (here `mdpreamble`). The package is deliberately independent of a particular visual style.

Chapter 4

The Document Preamble

4.1 mdpreamble.tex

One of the design goals of **md**layout is to separate the logical structure of a document from its visual appearance. The package itself provides semantic layout environments and document elements. The overall visual style, however, is intentionally left to the author. For this reason, every serious **md**layout project should begin with a well-designed document *preamble*.

The preamble is much more than a place for loading packages. It defines the typographic identity of the entire document, including fonts, colours, captions, hyperlinks, table styles, verbatim environments, warning boxes, indexes and many other global design decisions. Rather than prescribing a fixed appearance, **md**layout encourages authors to develop a consistent document design that reflects the purpose of their project. To simplify this process, the distribution contains the file

```
mdpreamble.tex
```

which is used without modification for producing this manual. The file is intended as a practical starting point rather than as a fixed template. Users are encouraged

to adapt it to their own requirements. Among other things, the example preamble demonstrates

- a complete font configuration using modern OpenType fonts,
- a consistent document colour palette,
- hyperlink and PDF configuration,
- table layouts,
- verbatim environments for source code,
- warning and information boxes,
- index generation,
- typography improvements,
- and numerous small refinements that contribute to a consistent overall appearance.

The supplied preamble deliberately follows the same design philosophy as the rest of **md**layout: semantic rather than visual descriptions. Instead of selecting arbitrary colours throughout the document, semantic design colours such as

```
mdColorPrimary
mdColorSecondary
mdColorAccent
mdColorAlert
```

are defined once and reused consistently. Likewise, document elements are described by their purpose rather than by their appearance. A compiler listing is typeset using the `listing` environment, while machine-generated output is displayed using `Printout`. Captions, warning boxes and hyperlinks all derive their appearance from the common design language established by the preamble.

Although the supplied `mdpreamble.tex` is fully functional, it is not intended to be the only possible solution. It should be regarded as a living example of how a complete technical document can be organized. Authors are encouraged to extend, simplify or completely redesign the preamble to match the needs of their own projects.

This manual itself demonstrates that approach: every page of the documentation is produced using the accompanying `mdpreamble.tex`. Consequently, every design decision described in the following chapters can immediately be studied in a real document and adapted for other projects.

4.2 Testing Mathematical Typesetting

If a technical document contains mathematical expressions, it can be useful to include a small formula test while setting up the document preamble. Such a test makes it possible to verify at an early stage whether the selected text and mathematics fonts work together correctly and whether all required mathematical symbols are available.

A suitable test should contain different kinds of mathematical notation: Latin and Greek characters, superscripts and subscripts, fractions, roots, operators, delimiters, and more complex displayed equations. The following examples provide a compact visual test of the mathematical typesetting configured in the standard **md**layout preamble ”`mdpreamble.tex`”.

A very simple example is Einstein’s mass–energy relation:

$$E = mc^2. \tag{4.1}$$

Here, E denotes energy, m denotes mass, and c is the speed of light. Apart from its physical meaning, this formula provides a useful typographical test because it combines mathematical variables with a superscript.

Fractions, roots, and Greek letters can be tested, for example, with the quadratic formula:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}. \tag{4.2}$$

The formula gives the solutions of a quadratic equation ($ax^2 + bx + c = 0$). From a typesetting perspective, it tests subscripts, a fraction, the `\pm` symbol, a square root, and superscripts within the same expression.

A somewhat more demanding example is the Gaussian integral:

$$\int_{-\infty}^{\infty} e^{-x^2}\,\mathrm{d}x = \sqrt{\pi}. \tag{4.3}$$

Its value is the square root of π. This expression additionally tests integral signs, limits, the infinity symbol, exponential notation, Greek characters, and the spacing around the differential $\mathrm{d}x$.

Finally, a formula containing several different mathematical structures can be used as a more comprehensive test:

$$f(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!}\, x^{2n+1}. \tag{4.4}$$

This is the Taylor series of the sine function,

$$f(x) = \sin x. \tag{4.5}$$

It provides a convenient test for summation symbols and their limits, fractions, factorials, parentheses, superscripts, and mathematical functions.

Einstein's field equations are a set of ten interconnected mathematical formulas in Albert Einstein's general theory of relativity that show how mass, energy, and momentum curve the fabric of spacetime

$$R_{\mu\nu} - \frac{1}{2}R\,g_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4}T_{\mu\nu}. \tag{4.6}$$

Maxwell's equations in standard vector differential form consist of four partial differential equations relating electric and magnetic fields to their sources

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}, \tag{4.7}$$

$$\nabla \cdot \mathbf{B} = 0, \tag{4.8}$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}, \tag{4.9}$$

$$\nabla \times \mathbf{B} = \mu_0\mathbf{J} + \mu_0\varepsilon_0\frac{\partial \mathbf{E}}{\partial t}. \tag{4.10}$$

Maxwell's equations in integral form relate electric and magnetic fields to their sources (charges and currents) over geometric surfaces and volumes

$$\oiint_{\partial V} \mathbf{E} \cdot d\mathbf{A} = \frac{Q_{\text{enc}}}{\varepsilon_0}, \tag{4.11}$$

$$\oiint_{\partial V} \mathbf{B} \cdot d\mathbf{A} = 0, \tag{4.12}$$

$$\oint_{\partial A} \mathbf{E} \cdot d\boldsymbol{\ell} = -\frac{d}{dt}\iint_A \mathbf{B} \cdot d\mathbf{A}, \tag{4.13}$$

$$\oint_{\partial A} \mathbf{B} \cdot d\boldsymbol{\ell} = \mu_0 I_{\text{enc}} + \mu_0\varepsilon_0\frac{d}{dt}\iint_A \mathbf{E} \cdot d\mathbf{A}. \tag{4.14}$$

A particularly compact example is provided by quantum electrodynamics (QED). Its dynamics can be described by the QED Lagrangian density

$$L_{\text{QED}} = \bar{\psi}\left(i\gamma^\mu D_\mu - m\right)\psi - \frac{1}{4}F_{\mu\nu}F^{\mu\nu}, \qquad D_\mu = \partial_\mu + ieA_\mu. \tag{4.15}$$

Here, ψ denotes the Dirac field describing electrically charged spin-$\frac{1}{2}$ matter, while $\bar{\psi}$ is its Dirac adjoint. The matrices γ^μ are the Dirac gamma matrices, m denotes the mass of the matter field, and e its electric charge. The covariant derivative D_μ couples the matter field to the electromagnetic four-potential A_μ.

The electromagnetic field itself is represented by the field-strength tensor

$$F_{\mu\nu} = \partial_\mu A_\nu - \partial_\nu A_\mu. \tag{4.16}$$

Expanding the covariant derivative makes the interaction between matter and the electromagnetic field explicit:

$$L_{\text{QED}} = \bar{\psi}\left(i\gamma^\mu\partial_\mu - m\right)\psi - e\,\bar{\psi}\gamma^\mu A_\mu\psi - \frac{1}{4}F_{\mu\nu}F^{\mu\nu}. \tag{4.17}$$

The three terms describe the free Dirac field, its interaction with the electromagnetic field, and the free electromagnetic field, respectively. This single expression therefore provides a useful typographical test for Greek letters, superscripts and subscripts, fractions, operators, and mathematical spacing.

Matrices provide another useful test because they combine large delimiters with numbers and mathematical operators. For example, multiplying two 2×2 matrices gives

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}. \tag{4.18}$$

Each element of the resulting matrix is obtained by multiplying the corresponding elements of a row of the first matrix with those of a column of the second matrix and adding the products.

The same operation can be written symbolically as

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x & y \\ z & w \end{pmatrix} = \begin{pmatrix} ax + bz & ay + bw \\ cx + dz & cy + dw \end{pmatrix}. \tag{4.19}$$

For general matrices A and B, their product $C = AB$ is defined component by component by

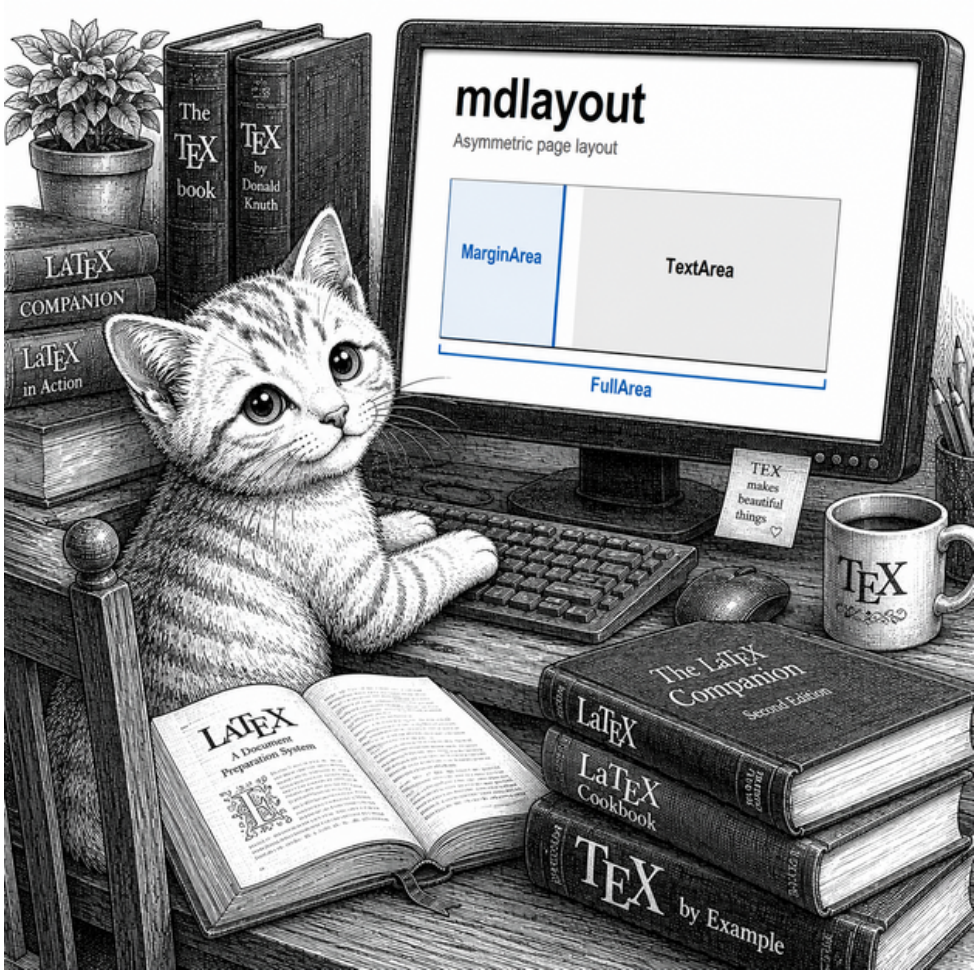
$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}. \tag{4.20}$$

Thus, the element c_{ij} is the scalar product of row i of A and column j of B. Besides being the general definition of matrix multiplication, this expression is particularly useful for testing summation symbols, limits, indices, and the relative appearance of text and mathematical numerals.

These examples are not intended as a mathematical or physical tutorial. Their primary purpose here is to provide a quick visual check of the mathematical configuration established in the document preamble. In particular, attention should be paid to the appearance of variables, Greek letters, operators, delimiters, superscripts and subscripts, and to whether the mathematics font harmonizes with the surrounding text font.

Chapter 5

The Layout Model



5.1 Fundamental Areas

The page layout is built upon two physical areas:

MarginArea The usable column to the left of the normal text column.

TextArea The normal KOMA-Script text column used for continuous prose.

The two areas are separated by the gutter. Together they form the **FullArea**:

☞ **MarginArea + Gutter + TextArea = FullArea**

The normal text column remains physically at the same horizontal position on odd and even pages. The layout is therefore asymmetric but not mirrored.

5.2 TextArea

The **TextArea** is the normal document area. It is used automatically for ordinary paragraphs and does not require a dedicated environment. Its width is available as:

```
\mdTextWidth
```

and remains identical to the calculated KOMA-Script text width at the time the layout is initialized.

5.3 MarginArea

The **MarginArea** occupies the usable left column and excludes the gutter. Its width is controlled by the package option:

```
marginwidth=<dimension>
```

and is available in the document as:

```
\mdMarginWidth
```

The MarginArea is intended for small accompanying figures or other explicitly positioned material. It is not intended for text, although it is possible to write there.

✖ Avoid too much content in the MarginArea.

5.4 HalfArea

The **HalfArea** extends the normal text column into half of the wide offset. Its total width is:

```
\mdHalfWidth
```

and the horizontal extension is:

```
\mdHalfOffset
```

The HalfArea is suitable for material that requires more width than ordinary text but does not require the complete FullArea.

5.5 FullArea

The **FullArea** uses the complete available layout width from the left edge of the MarginArea to the right edge of the TextArea. Its width is available as:

```
\mdFullWidth
```

The horizontal distance between the left edges of TextArea and FullArea is:

```
\mdWideOffset
```

with:

```
\mdWideOffset = \mdMarginWidth + \mdGutterWidth
```

5.6 Synchronized Margin and Text Areas

5.6.1 Synchronizing content by \SyncMarginAndTextArea

Sometimes an object in the margin is not independent information but refers directly to a specific object in the main text area. Typical examples are

- a short warning or comment,
- a symbol,
- a keyword or label,
- a small explanatory note,
- or a graphical marker referring to a paragraph beside it.

For short and simple content, **md**layout provides the command

```
\SyncMarginAndTextArea{<margin object>}{<text object>}
```

The first argument is placed in the `MarginArea`, the second in the `TextArea`. Both objects begin at exactly the same vertical position. For example:

```
\SyncMarginAndTextArea
{This paragraph is not important!}
{\emph{\lipsum[1][1-5]}}
```

This paragraph is not important!

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque.

The command is intentionally implemented using two synchronized top-aligned minipages. It is not intended to provide a general two-column text layout. **md**layout has no need to duplicate the many existing solutions for ordinary multicolumn typesetting.

The command form is particularly convenient when both objects are short. For longer or structurally more complex material, however, passing the complete contents as command arguments can make the document source difficult to read. For such cases, **md**layout also provides an environment-based form.

5.6.2 Synchronized content by the *SynchronizedMarginAndTextArea* environment

For longer or more complex synchronized content, the *environment*

```
\begin{SynchronizedMarginAndTextArea}
 \begin{TextInMarginArea}
  <margin content>
 \end{TextInMarginArea}
 \begin{TextInTextArea}
  <text content>
 \end{TextInTextArea}
\end{SynchronizedMarginAndTextArea}
```

provides the same basic layout principle as `\SyncMarginAndTextArea`, but without requiring the contents to be passed as command arguments. The two inner environments explicitly describe the semantic role of their contents: `TextInMarginArea` contains the material to be placed in the `MarginArea`, while `TextInTextArea` contains the associated main text.

For example, the cat illustration and the accompanying text beginning with "*Such code describes the implementation rather than the intention.*" on Page 10 were typeset using:

```
\begin{SynchronizedMarginAndTextArea}
 \begin{TextInMarginArea}
  \PDFImage[width=\mdMarginWidth]{mdlayout-tixi-measuring}
 \end{TextInMarginArea}
 \begin{TextInTextArea}
  Such code describes the implementation rather than the intention.
  The value \texttt{-37mm} has no semantic meaning.
  A reader cannot determine why this particular value was chosen or
  what the author intended to achieve.
  ...
 \end{TextInTextArea}
\end{SynchronizedMarginAndTextArea}
```

> ☞ The `SynchronizedMarginAndTextArea` *environment* is, in contrast to the *command form* `\SyncMarginAndTextArea{...}{...}`, robust against *verbatim* content. The latter does not accept *verbatim* content in its arguments.

Chapter 6

Area Environments

6.1 HalfArea

The `HalfArea` environment extends the current text block into half of the MarginArea and gutter offset.

6.1.1 Syntax

```
\begin{HalfArea}
 ...
\end{HalfArea}
```

6.1.2 Example

```
\begin{HalfArea}
 This paragraph uses more horizontal space than ordinary text but does not
 occupy the complete FullArea.
\end{HalfArea}
```

The environment is heading-aware. Section, subsection, and other supported KOMA-Script headings placed inside the environment are compensated automatically and remain correctly aligned.

6.2 FullArea

The `FullArea` environment extends content over the complete available page width.

6.2.1 Syntax

```
\begin{FullArea}
 Very long line ...
\end{FullArea}
```

6.2.2 Example


```
\begin{FullArea}
 \begin{Center}
  \PDFImage[width=0.7\linewidth]{mdlayout-tixi-juggling}
 \end{Center}
\end{FullArea}
```

Inside the environment, `\linewidth` reflects the widened area supplied by `adjustwidth`. The environment is suitable for:

- wide figures,
- wide tables,
- listings,
- terminal output,
- title pages,
- wide chapter content.

Commands that perform their own page breaking, especially `\part` and `\chapter`, should generally be placed outside the environment. Their following content may then be enclosed in `FullArea`.

6.3 DirectoryArea

The DirectoryArea environment is a semantic alias of HalfArea.

6.3.1 Syntax

```
\begin{DirectoryArea}
 ...
\end{DirectoryArea}
```

6.3.2 Typical Use

```
\begin{DirectoryArea}
 \tableofcontents
 \clearpage
 \listoffigures
 \clearpage
 \listoftables
\end{DirectoryArea}
```

It is intended for:

- table of contents,
- list of figures,
- list of tables,
- bibliography,
- glossary,
- index.

Although its implementation is identical to HalfArea, its name expresses the intended purpose of the content.

6.4 MarginArea

The MarginArea environment places ordinary material into the usable left column.

6.4.1 Syntax

```
\begin{MarginArea}
 ...
\end{MarginArea}
```

The environment excludes the gutter and occupies only the configured MarginArea width. It is intended for explicitly positioned material. For figures synchronized with surrounding text, use MarginFigure instead.

6.5 Convenience Environments

The package also provides the following convenience environments:

`FullCenter` A centred FullArea.

`MarginCenter` A centred MarginArea.

`widearea` Compatibility alias for `FullArea`.

`widecenter` Compatibility alias for `FullCenter`.

Lower-case aliases are also provided for selected area environments:

```
halfarea
fullarea
directoryarea
marginarea
margincenter
```

Chapter 7

Margin Figures

7.1 Purpose

The `MarginFigure` environment places a small, referenceable, non-floating figure into the fixed MarginArea. It does not use LaTeX's `\marginpar` mechanism because the MarginArea is not an outer page margin. It is a dedicated layout column inside the FullArea.

7.2 Syntax

```
\begin{MarginFigure}[<font declaration>]
 <figure contents>
 \caption{<caption>}
 \label{<label>}
\end{MarginFigure}
<associated text>
```

The optional argument controls the caption font size. The default is:

```
\footnotesize
```

Other declarations may be used, for example:

```
\begin{MarginFigure}[\small]
```

or:

```
\begin{MarginFigure}[\normalsize]
```

7.3 Caption Formatting

MarginFigure captions use a compact format suitable for the narrow column. The caption is:

- set without hanging indentation,

- left aligned,
- allowed to wrap across the complete MarginArea width,
- formatted using the selected optional font size.

Global caption colours and label styles remain available.

7.4 Example

Figure 7.1: Tixi handles MarginArea, FullArea, and TextArea in a playful manner.

The top edge of MarginFigure 7.1 is aligned approximately with the first line of the following text. There should be no empty paragraph between `\end{MarginFigure}` and the associated text.

```
\begin{MarginFigure}[\footnotesize]
 \PDFImage[width=\linewidth]{mdlayout-tixi-juggling}
 \caption{Tixi handles MarginArea, FullArea, and TextArea
  in a playful manner.}
 \label{fig:mdlayout-tixi-juggling}
\end{MarginFigure}
The top edge of MarginFigure \ref{fig:mdlayout-tixi-juggling}
is aligned ...
```

7.5 Non-floating Behaviour

`MarginFigure` is intentionally non-floating. It remains at the source position chosen by the author. No automatic vertical repositioning is performed. This has the following consequences:

- the author must select a sufficiently long accompanying text passage;
- the figure must not extend below the page boundary;
- following headings must not overlap the figure;
- multiple MarginFigures must not occupy the same vertical region.

These conditions are part of the semantics of a fixed, non-floating object. If automatic placement is required, use the standard `figure` environment or `widefigure`.

7.6 Use Inside Lists

MarginFigure compensates for the current accumulated list indentation. It can therefore be placed inside environments such as:

- `description`,
- `itemize`,
- `enumerate`.

Its horizontal position remains fixed at the left edge of the MarginArea.

Chapter 8

Wide Figures and Tables

8.1 widefigure

The `widefigure` environment combines the standard floating `figure` environment with `FullArea`.

8.1.1 Syntax

```
\begin{widefigure}[<placement>]
 ...
\end{widefigure}
```

The placement specification is:

`tbp`

8.1.2 Example

```
\begin{widefigure}[htbp]
 \includegraphics[width=\linewidth]{large-screen.png}
 \caption{Full-width application screen}
 \label{fig:large-screen}
\end{widefigure}
```

The environment remains a normal LaTeX float. Its placement is therefore controlled by LaTeX.

8.2 widetable

The `widetable` environment is the table equivalent of `widefigure`.

8.2.1 Syntax

```
\begin{widetable}[<placement>]
 ...
\end{widetable}
```

8.2.2 Example

```
\begin{widetable}[htbp]
 \begin{tabular}{lll}
  ...
 \end{tabular}
 \caption{Wide command table}
 \label{tab:wide-command-table}
\end{widetable}
```

8.3 widelongtable

The `widelongtable` environment provides FullArea geometry for longtable-based environments such as `longtable` and `xltabular`.

8.3.1 Syntax

```
\begin{widelongtable}
 \begin{xltabular}{\linewidth}{...}
  ...
 \end{xltabular}
\end{widelongtable}
```

8.3.2 Example

```
\begin{widelongtable}
 \begin{xltabular}{\linewidth}{@{}lX@{}}
  Command & Description \\
  \hline
  ...
 \end{xltabular}
\end{widelongtable}
```

The environment locally configures the internal longtable alignment so that the table begins at the left edge of the FullArea. It also compensates the caption-package offset that would otherwise move a longtable caption too far to the right. The package `longtable` or `xltabular` must be loaded before using this environment.

8.4 Recommended Table Environment

For new tables, the semantic `ReferenceTable` environment described in Chapter 15 is recommended. It expresses the purpose and properties of a table through

options such as its area, column roles, headers, and alignment, without requiring the document author to construct the layout from low-level column specifications.

The `widetable` and `widelongtable` environments are retained primarily for compatibility. They are useful when an existing document already uses the classical `tabular`, `longtable`, or `xltabular` environments, or when direct access to those environments is specifically required. In such cases, they extend the classical table into the FullArea without requiring it to be rewritten as a `ReferenceTable`.

Chapter 9

Headings, Headers, and Footers

9.1 Wide Headings

When the option

```
wideheadings=true
```

is enabled, KOMA-Script headings are extended to the FullArea. The package supports:

- parts,
- chapters,
- unnumbered chapter-level headings,
- sections,
- lower section levels.

Numbering, prefix handling, fonts, language, and semantic heading behaviour remain under KOMA-Script control. Only the horizontal geometry and optional rules are modified.

9.2 Heading Compensation Inside Areas

The area environments communicate their horizontal offset to the heading formatter. This prevents headings from being shifted twice when they occur inside:

- `HalfArea`,
- `DirectoryArea`,
- `FullArea`.

9.3 Chapter and Section Rules

Optional decorative rules may be applied to chapter and section headings. The relevant package options are:

```
chapterrule=<dimension>
sectionrule=<dimension>
headingrulecolor=<colour>
chapterrulegap=<dimension>
sectionrulegap=<dimension>
```

A rule width of `0pt` disables the corresponding rule.

9.4 Wide Headers and Footers

When

```
wideheadfoot=true
```

is enabled, page headers and footers extend across the FullArea. The package installs a `scrlayer-scrpage` page style. In two-sided documents:

- even pages show the page number on the left and the running heading on the right;
- odd pages show the running heading on the left and the page number on the right.

Chapter-opening pages use `plain.scrheadings` with a centred page number in the footer.

Chapter 10

Package Options

The **md**layout package supports the options given in Table 10.1.

Table 10.1: Package Options

Option	Default	Description
marginwidth	38mm	Width of the usable MarginArea.
gutter	5mm	Horizontal separation between MarginArea and TextArea.
rightmargin	20mm	Physical right page margin. If omitted, the calculated KOMA-Script right margin is retained.
bottommargin	30mm	Physical distance between the lower text edge and the paper. If set to an empty value, the calculated KOMA-Script *text height* is retained.
wideheadings	true	Moves supported KOMA-Script headings to the left edge of the FullArea.
wideheadfoot	true	Extends page headers and footers over the FullArea.
headsepline	0.8pt	Thickness of the separator below the page header. Use false to disable it.
plainheadsepline	false	Shows the header separator on plain chapter-opening pages.
chapterrule	2.0pt	Thickness of the rule used with chapter headings.
sectionrule	1.4pt	Thickness of the rule used with section headings.
headingrulecolor	black	Colour used for chapter and section heading rules.
chapterrulegap	0.55\baselineskip	Vertical separation between chapter rule and chapter heading; here chapterrulegap=7.48004pt.
sectionrulegap	0.35\baselineskip	Vertical separation associated with the section rule; here sectionrulegap=4.76007pt.
showlayout	false	Writes the calculated layout dimensions to the log file.
articletitlerule	3.0pt	Thickness of the title rules used for scrartcl class.
articletitlerulegap	0.55\baselineskip	Vertical separation between title rules used for scrartcl; here articletitlerulegap=7.48004pt.
widetitle	true	Controls whether document titles use the full horizontal page area.
minleftmargin	10mm	Minimum physical distance between the left edge of the paper and the FullArea.
minmarginwidth	20mm	Minimum width of the MarginArea when the layout has to be adapted to a narrow paper format.

continued…

Option	Default	Description
`mintextwidth`	`80mm`	Minimum width of the `TextArea` during automatic layout adaptation.

☞ **Implementation Note:**
The table itself is implemented using `ReferenceTable`; for details, see Chapter 15. Its first column reproduces option names verbatim, while its second column uses normal LaTeX processing in typewriter style. The relevant part of its definition[a] is:

```
\begin{ReferenceTable}[
        area=full,
        caption={Package Options},
        label={tab:PackageOptions},
        columns=3,
        verbatimcolumn=1,
        typewritercolumn=2,
        xcolumn=3
        ]{Option}{Default}{Description}

        marginwidth & \mdDefaultMarginwidth & Width of ...
\end{ReferenceTable}
```

[a] The values in the second column are not duplicated as literal values in the documentation source. Instead, they are obtained from public definitions provided by **md**layout itself. For example, `\mdDefaultMarginwidth` expands to the default value of the `marginwidth` option.

Remarks:

- With `widetitle=true`, both titles generated by `\maketitle` and explicit `titlepage` environments use the `FullArea`. Consequently, an explicit `FullArea` environment should not be nested inside a `titlepage` environment. With `widetitle=false`, the original KOMA-Script title layout is retained.
- `articletitlerule=<length>` sets the thickness of the horizontal rules surrounding the document title in the `scrartcl` class. The rules extend across the `FullArea` and visually relate the article title to the ruled section headings used by **md**layout. A value of `0pt` *disables* the title rules.
- The following three `min...` parameters control automatic layout adaptation for small paper sizes, such as *DIN A5 paper*. If the requested layout does not fit on the available paper width, **md**layout first reduces the `MarginArea` down to `minmarginwidth`. If this is not sufficient, the `TextArea` is subsequently reduced down to `mintextwidth`. The `gutter`, `rightmargin`, and the minimum physical left margin (`minleftmargin`) remain unchanged. If a valid layout cannot be obtained without violating these minimum dimensions, **md**layout reports an error.
  For DIN A5 paper, a balanced layout can, for example, be obtained with:

  ```
  \usepackage[
  rightmargin=15mm,
  minleftmargin=15mm
  ```

```
]{mdlayout}
```

This gives the `FullArea` equal margins of 15 mm on the left and right while retaining the automatic adaptation of the `MarginArea` and `TextArea`.

Chapter 11

Public Dimensions and Commands

11.1 Public Dimensions

The package exposes the calculated layout dimensions listed in Table 11.1. This table was typeset by:

```
\begin{ReferenceTable}[
        caption={Public Dimensions},
        label={tab:PublicDimensions},
        area=text,
        columns=2,
        verbatimcolumn=1,
        xcolumn=2
        ]{Dimension}{Meaning}
        \mdTextWidth  &  Width of the normal TextArea. \\
        ...
\end{ReferenceTable}
```

Table 11.1: Public Dimensions

Dimension	Meaning
\mdTextWidth	Width of the normal TextArea.
\mdFullWidth	Complete width of MarginArea, gutter, and TextArea.
\mdMarginWidth	Width of the usable MarginArea.
\mdGutterWidth	Width of the gutter.
\mdWideOffset	Horizontal distance from the left edge of TextArea to the left edge of FullArea.
\mdHalfOffset	Half of the wide offset.
\mdHalfWidth	Width of TextArea plus half the wide offset.
\mdRightMargin	Physical right page margin.
\mdBottomMargin	Physical lower page margin.
\mdFullLeft	Physical position of the left edge of FullArea measured from the left paper edge.
\mdFullRight	Physical position of the right edge of FullArea.
\mdMarginLeft	Physical position of the left edge of MarginArea.
\mdMarginRight	Physical position of the right edge of MarginArea.

continued…

Dimension	Meaning
\mdTextLeft	Physical position of the left edge of TextArea.
\mdTextRight	Physical position of the right edge of TextArea.

11.2 Compatibility Dimensions

The package also provides compatibility names:

```
\fullwidth
\marginwidth
\gutterwidth
\wideoffset
\megatextindent
\textindent
```

New documents should preferably use the explicit \md... names.

11.3 \ShowMDLayout

The command

```
\ShowMDLayout
```

prints a table of the calculated dimensions inside the document. It is useful during layout development and diagnostics.

11.4 \mdlayoutrecalculate

The command

```
\mdlayoutrecalculate
```

recalculates the layout using the current text width and side margins. It may be useful after document-class or layout changes that occur after the package has been loaded.

Small paper formats. On smaller paper formats, **md**layout automatically reduces the MarginArea and, if necessary, the TextArea, while respecting their configured minimum widths. For A5 paper, a balanced layout can for example be obtained with:

```
\documentclass[a5paper]{scrbook}
\usepackage[
rightmargin=15mm,
minleftmargin=15mm
]{mdlayout}
```

This gives the `FullArea` equal physical margins of 15 mm on the left and right while retaining the adaptive behaviour of **md**layout for the `MarginArea` and `TextArea`.

11.5 Convenience Commands

The package also provides:

`\fullcenter {...}` Centres its argument inside `FullArea`.

`\marginbox {...}` Places its argument inside `MarginArea`.

`\fullrule [<thickness>]` Draws a horizontal rule across `FullArea`. The default thickness is `0.4pt`.

`\begin {typewriter}...\end {typewriter}` Typesets its contents in a verbatim-like typewriter layout while retaining normal LaTeX command processing. Unlike the standard `verbatim` environment, commands inside `typewriter` are evaluated. Thus, for example,

```
\begin{typewriter}
 rightmargin=\mdDefaultRightmargin
\end{typewriter}
```

displays the current definition of `\mdDefaultRightmargin` in typewriter style rather than printing the command name literally. The environment is intended for code-like documentation in which selected LaTeX definitions should remain active.

`\company{<company>}` Specifies optional company or institutional information associated with the document.

The company is included in the document title information. In `scrbook` and `scrreprt`, it is additionally displayed at the physical left side of the footer below the `MarginArea`. The existing page header, including the page number, is not modified.

If `\company` is not specified, no company information is displayed and the footer remains unchanged.

Chapter 12

Version Information

12.1 Package Metadata

The package provides the following public macros:

```
\mdlayoutPackageName
\mdlayoutDate
\mdlayoutVersion
\mdlayoutRevisionNumber
```

For version 0.8.3 they expand conceptually to:

```
mdlayout
2026/08/06
0.8.3
000803
```

12.2 Revision Number Encoding

The numeric revision number is generated automatically from the semantic version. The encoding follows:

```
xx.yy.zz -> xxyyzz
```

Examples:

```
0.7.2  -> 000702
0.8.3  -> 000803
1.0.0  -> 010000
12.3.4 -> 120304
```

The major component may contain more than two digits. The minor and patch components are encoded using two digits. Leading zeros do not affect numerical comparisons in TeX.

12.3 Conditional Version Tests

The command

```
\IfMDLayoutRevisionAtLeast{<revision>}{<code>}
```

executes the supplied code if the installed package revision is at least the requested value.
Example:

```
\IfMDLayoutRevisionAtLeast{000800}{%
 This feature requires mdlayout 0.8.0 or later.
}
```

A direct numerical test is also possible:

```
\ifnum\mdlayoutRevisionNumber<000803
\PackageError{mypackage}
{mdlayout >= 0.8.3 required}
{}
\fi
```

Chapter 13

Examples

13.1 Math-mode symbols

Here we have a wide table of LaTeX 2ε math symbols in a `tabular` environment. This `tabular` is embedded in a `FullArea`:

≤	\leq	≥	\geq	≠	\neq	≈	\approx	×	\times	÷	\div	±	\pm	·	\cdot
ˆ	^{\circ}	∘	\circ	′	\prime	⋯	\cdots	∞	\infty	¬	\neg	∧	\wedge	∨	\vee
⊃	\supset	∀	\forall	∈	\in	→	\rightarrow	⊂	\subset	∃	\exists	∉	\notin	⇒	\Rightarrow
∪	\cup	∩	\cap	∣	\mid	⇔	\Leftrightarrow	$\dot{a}$	\dot a	$\hat{a}$	\hat a	$\bar{a}$	\bar a	$\tilde{a}$	\tilde a
α	\alpha	β	\beta	γ	\gamma	δ	\delta	ϵ	\epsilon	ζ	\zeta	η	\eta	ε	\varepsilon
θ	\theta	ι	\iota	κ	\kappa	ϑ	\vartheta	λ	\lambda	μ	\mu	ν	\nu	ξ	\xi
π	\pi	ρ	\rho	σ	\sigma	τ	\tau	υ	\upsilon	ϕ	\phi	χ	\chi	ψ	\psi
ω	\omega	Γ	\Gamma	Δ	\Delta	Θ	\Theta	Λ	\Lambda	Ξ	\Xi	Π	\Pi	Σ	\Sigma
Υ	\Upsilon	Φ	\Phi	Ψ	\Psi	Ω	\Omega								

13.2 Long commands

Here is a Unix example for process monitoring:

```
ps -eo user,pid,ppid,%cpu,%mem,etime,command \
| grep -v grep \
| grep "java\|python\|go"
```

It was typeset using the following commands:

```
\begin {verbatim}

 ps -eo user,pid,ppid,%cpu,%mem,etime,command \
 | grep -v grep \
 | grep "java\|python\|go"

\end {verbatim}
```

Here is the same example printed in one line.

```
ps -eo user,pid,ppid,%cpu,%mem,etime,command | grep -v grep | grep "java\|python\|go"
```

This was typeset by:

```
\begin {wideverbatim}

 ps -eo user,pid,ppid,%cpu,%mem,etime,command | grep -v grep | grep "java\|python\|go"
```

```
\end {wideverbatim}
```

Note: The lines of LaTeX code above (`\begin{wideverbatim}...`) were typeset using **FullArea**. The commands used for this can be found in the source file `mdlayout-doc.tex` and in the following listing.

mdlayout-doc.tex

```
\begin{FullArea}
        \latex{\begin{wideverbatim}}
                \begin{verbatim}
                        ps -eo user,pid,ppid,%cpu,%mem,etime,command | grep -v grep | grep "java\|python\|go"
                \end{verbatim}
        \latex{\end{wideverbatim}}
\end{FullArea}
```

The listing has been typeset by:

```
\begin{listing}[label={mdlayout-doc.tex}]
 ...
\end{listing}
```

13.3 Using \SyncMarginAndTextArea for Annotations

The purpose of **\SyncMarginAndTextArea** is to establish a *fixed* semantic relationship between a relatively small object in the margin and the corresponding content in the text area. Consequently, the complete synchronized object is kept together and is not broken across pages. This is normally desirable: a margin annotation that becomes separated from the paragraph, figure, warning, or other object to which it refers would lose its meaning. For this reason, both arguments should normally remain *reasonably short*. If a large amount of parallel text is required, a genuine multicolumn or table-based layout is more appropriate.

Note:

`\SyncMarginAndTextArea` is not a two-column environment.
It is intended to establish a fixed semantic relationship between a small object in the margin and its associated object in the text area. Both objects always start on the same line and remain on the same page.

This *Note* has been typeset by:

```
\SyncMarginAndTextArea
{%
 \textbf{Note:}
}
{%
 \textbf{\texttt{\textbackslash SyncMarginAndTextArea} is not a
  two-column environment.}

 It is intended to establish a fixed semantic relationship between a
 small object in the margin and its associated object in the text
```

```
 area. Both objects always start on the same line and remain on the
 same page.
}
```

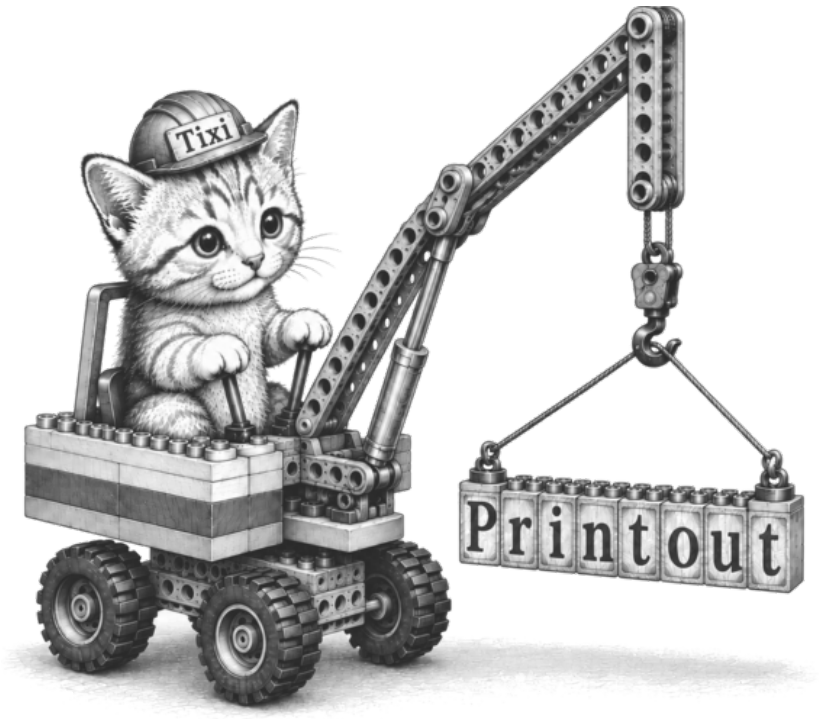
The purpose of this command is not to place two independent text columns side by side. Instead, it represents a single semantic document element whose content happens to occupy both the margin area and the text area.

In the example above, we used `\mdSeeRight` to draw attention to the text:

```
\SyncMarginAndTextArea{\mdSeeRight}
{%
 The purpose of this command is not to place two independent
 text columns side by side. Instead, it represents a single
 semantic document element whose content happens to occupy
 both the margin area and the text area.
}
```

Chapter 14

The `Printout` Environment



The `Printout` environment is designed for displaying machine-generated output such as system logs, compiler listings, terminal sessions, spool files, printer output, or reports produced by external programs.

Unlike traditional source-code listings, a printout represents an already existing document whose visual structure is part of its meaning. The original line lengths, column positions and spacing should therefore be preserved as accurately as possible.

Consequently, `Printout` should not be regarded as another listing environment. It is a semantic environment for typesetting computer printouts.

14.1 Printout versus Listings

Packages such as `listings` or `minted` are intended for typesetting source code. They provide syntax highlighting, language support and formatting facilities that

help understanding a program. Machine-generated output has completely different requirements. A compiler listing, system log or printer spool already possesses its final formatting. Reformatting the text may destroy the alignment of columns or even change the interpretation of the output.

For this reason, the Printout environment never attempts to reformat its contents. Instead, it preserves the original character layout while adapting the font size only if required.

14.2 Automatic Column Fitting

Perhaps the most important feature of Printout is that the logical width of the output is specified instead of a font size. The option

```
fitcolumns=<n>
```

declares that at least n monospaced characters shall fit into the available print area.
For example,

```
\begin{Printout}[
 form=plain,
 fitcolumns=132,
 fontsize=\normalsize
 ]
 ...
\end{Printout}
```

requests normal-sized text whenever possible. Only if 132 columns would no longer fit into the available width is the font automatically reduced. Thus the author specifies the semantic width of the printout instead of experimenting with font sizes. This behaviour is particularly useful for compiler listings, JES output, terminal logs and similar documents whose logical width is fixed.

14.3 Printout Forms

The appearance of the printout is controlled by the form= option. A form is simply a predefined collection of ordinary Printout options. All values supplied after form= override the preset values. The package currently provides the following forms.

plain Produces plain machine output without any graphical paper decoration. This form is intended as a general-purpose verbatim printout while still benefiting from automatic column fitting.

simple Displays the traditional green-bar paper without tractor-feed holes or line numbers.

`green` Displays continuous green-bar paper with tractor-feed holes and functional line numbers.

`blue` Equivalent to `green`, but using blue paper stripes.

`ibm1403` Emulates the appearance of a classical IBM 1403 line-printer listing. This preset uses 60 printable lines per paper page, three-line green bars and a smaller font size.

A typical example is

```
\begin{Printout}[form=ibm1403]
 ...
\end{Printout}
```

Individual properties may easily be overridden.

```
\begin{Printout}[
 form=green,
 paperheight=200
 ]
 ...
\end{Printout}
```

The printout keeps the appearance of the green-bar form while using virtual paper pages consisting of 200 text lines.

14.4 Paper Geometry

Unlike ordinary verbatim environments, `Printout` models the geometry of continuous fan-fold paper. Depending on the selected form, the following elements may be displayed.

- coloured paper bands,
- paper borders,
- virtual tear-off perforations,
- tractor-feed holes,
- functional line numbers.

All geometric elements are derived from the actual height and depth of the current text line. Consequently, changing the font size automatically preserves the visual proportions of the paper.

14.5 Functional Line Numbers

The optional line numbers are not merely decorative. Every physical output line receives an explicit number, making it easy to reference individual lines in

compiler listings, system logs or spool files. Since the numbering restarts on every virtual paper page, even extremely large printouts remain easy to navigate.

14.6 Virtual Continuous Paper

Real fan-fold paper consists of separate sheets connected by perforations. The option

```
paperheight=<lines>
```

defines the number of text lines per virtual sheet. Whenever the specified number of lines has been printed, a virtual tear-off perforation is inserted and the line numbering restarts with 1.
For example,

```
paperheight=60
```

reproduces the appearance of a traditional 60-line IBM line-printer listing, whereas

```
paperheight=999
```

creates virtually continuous paper suitable for long log files.

14.7 Examples

14.7.1 Printout Example: IBM 1403 line printer output with 132 chars

The following example shows a printout orientated on the vintage IBM line printer model 1403.

```
\begin{Printout}[form=ibm1403]
...
\end{Printout}
```

```
HH        HH  EEEEEEEEEEE  RRRRRRRRRR    CCCCCCCCCC     00000000        11       ZZZZZZZZZZZZ  ZZZZZZZZZZZZ
HH        HH  EEEEEEEEEEE  RRRRRRRRRRR  CCCCCCCCCCCC   0000000000      111       ZZZZZZZZZZZZ  ZZZZZZZZZZZZ
HH        HH  EE           RR        RR  CC        CC  00      0000     1111             ZZ            ZZ
HH        HH  EE           RR        RR  CC            00     00 00       11            ZZ            ZZ
HH        HH  EE           RR        RR  CC            00    00  00       11           ZZ            ZZ
HHHHHHHHHHHH  EEEEEEE      RRRRRRRRRRR   CC            00   00   00       11         ZZZZZZZ       ZZZZZZZ
HHHHHHHHHHHH  EEEEEEE      RRRRRRRRRRR   CC            00  00    00       11         ZZZZZZZ       ZZZZZZZ
HH        HH  EE           RR    RR      CC            00 00     00       11          ZZ            ZZ
HH        HH  EE           RR     RR     CC            0000      00       11          ZZ            ZZ
HH        HH  EE           RR      RR    CC        CC  000       00       11         ZZ            ZZ
HH        HH  EEEEEEEEEEE  RR       RR   CCCCCCCCCCCC   0000000000    1111111111   ZZZZZZZZZZZZ  ZZZZZZZZZZZZ
HH        HH  EEEEEEEEEEE  RR        RR   CCCCCCCCCC     00000000     1111111111   ZZZZZZZZZZZZ  ZZZZZZZZZZZZ


  JJJJJJJJJJ  OOOOOOOOOOO  BBBBBBBBBB     00000000      00000000     8888888888  777777777777       444          AAAAAAAAAA
  JJJJJJJJJJ  OOOOOOOOOOO  BBBBBBBBBBB   0000000000    0000000000  888888888888  77777777777       4444         AAAAAAAAAAA
```

```
      JJ      OO        OO  BB        BB  00      0000  00      0000  88        88  77      77        44 44            AA        AA
      JJ      OO        OO  BB        BB  00     00 00  00     00 00  88        88        77        44  44            AA        AA
      JJ      OO        OO  BB       BB   00    00  00  00    00  00   88      88        77        44   44            AA        AA
      JJ      OO        OO  BBBBBBBBB    00   00   00  00   00   00    88888888        77        44444444444         AAAAAAAAAAA
      JJ      OO        OO  BBBBBBBBB    00  00    00  00  00  00     88888888        77        444444444444        AAAAAAAAAAA
      JJ      OO        OO  BB       BB   00 00     00  00 00     00   88      88      77              44            AA        AA
JJ    JJ      OO        OO  BB        BB  0000      00  0000      00  88        88    77              44            AA        AA
JJ    JJ      OO        OO  BB        BB  000       00  000       00  88        88    77              44            AA        AA
JJJJJJJJ      OOOOOOOOOOOO  BBBBBBBBBBB   0000000000    0000000000   888888888888    77              44            AA        AA
 JJJJJJ       OOOOOOOOOOOO  BBBBBBBBBB     00000000      00000000     8888888888     77              44            AA        AA


****A  START  JOB    JOB00874     HERC01ZZ   Compress a pds   04.19.13 PM 07 Aug  SYS    HOST09     JOB    JOB00874     START  A****
****A  START  JOB    JOB00874     HERC01ZZ   Compress a pds   04.19.13 PM 07 Aug  SYS    HOST09     JOB    JOB00874     START  A****
****A  START  JOB    JOB00874     HERC01ZZ   Compress a pds   04.19.13 PM 07 Aug  SYS    HOST09     JOB    JOB00874     START  A****
****A  START  JOB    JOB00874     HERC01ZZ   Compress a pds   04.19.13 PM 07 Aug  SYS    HOST09     JOB    JOB00874     START  A****


J E S 2  J O B  L O G  --  B R A I N . R E P O R T / E V A L  S Y S T E M

16:19:13 JOB00874 IEF501I JOB HERC01ZZ 'Compress a pds' CLASS A - JES2 SUBMISSION STARTED
16:19:13 JOB00874 IEF600I PRE-VALIDATING JCL
16:19:13 JOB00874 IEF601I JCL PRE-VALIDATED
000001 //* SYS2.JCLLIB.COMPRESS.JCL
000002 //HERC01ZZ JOB (COMPRESS),'Compress a pds',CLASS=A,MSGCLASS=P
000003 //* USER=HERC01,PASSWORD=xxxxxxxx
000004 //*
000005 //*********************************************************************
000006 //*
000007 //*  Name: SYS2.JCLLIB(COMPRESS)
000008 //*
000009 //*  Desc: Compress a PDS
000010 //*  Step environment variable: DEBUG=TRUE
000011 //*********************************************************************
000012 //*
000013 //*
000014 //DELETE   EXEC PGM=IEFBR14
000015 //LIB2DEL  DD   DISP=DEL,
000016 //              DSN=HERC01.TEST.LOADLIB2,
000017 //              DSNTYPE=LIBRARY
000018 //SYSOUT   DD   SYSOUT=*
000019 //COMPRESS EXEC PGM=IEBCOPY,PARM='ENVAR("DEBUG=TRUE")/'
000020 //SYSPRINT DD   SYSOUT=*
000021 //SYSOUT   DD   SYSOUT=*
000022 //SYSUT1   DD   DISP=SHR,
000023 //              DSN=HERC01.TEST.LOADLIB
000024 //SYSUT2   DD   DISP=NEW,
000025 //              DSN=HERC01.TEST.LOADLIB2,
000026 //              DSNTYPE=LIBRARY
000027 //SYSIN    DD   *
000028 COPY INDD=SYSUT1,OUTDD=SYSUT2
000029 SELECT MEMBER=(MEMBER1)
000030 /*
000031 //
16:19:13 JOB00874 IRR010I USERID '' IS ASSIGNED TO THIS JOB.
16:19:13 JOB00874 IEF236I JOB HERC01ZZ STARTED - CLASS A - 2026-08-07 16:19:13
16:19:13 JOB00874 IEF237I ACCT=COMPRESS
16:19:13 JOB00874 $HASP373 HERC01ZZ MSGCLASS=P MSGLEVEL=(1,1) Compress a pds
16:19:13 JOB00874 IEF373I STEP/DELETE/STARTED AT 2026-08-07 16:19:13
16:19:13 JOB00874 IEF574I ENVAR LIB2DEL=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB2
16:19:13 JOB00874 IEC043I DELETING PDSE LIBRARY (HERC01.TEST.LOADLIB2)
16:19:13 JOB00874 IEC032I DELETE HERC01.TEST.LOADLIB2 (PDSE DATASET DELETED)
16:19:13 JOB00874 IEC231I USING DEFAULT TIMEOUT 20 SECONDS (empty TIME string)
------ SYSOUT BEGIN ------
------- SYSOUT END -------
16:19:13 JOB00874 IEF374I STEP/DELETE/ENDED AT 2026-08-07 16:19:13 RC=0000
16.19.13 JOB00874 - ============================================================================================================
16.19.13 JOB00874 -                          REGION MAX CPU     ---- STEP TIMINGS ----                ----PAGING COUNTS----
16.19.13 JOB00874 - STEPNAME PGMNAME    CC    USED    LOAD     CPU TIME  ELAPSED TIME   EXCP  SERV  PAGE  SWAP  VIO SWAPS
16.19.13 JOB00874 - DELETE   HERC01ZZ 0000  1024K      0%  00:00:00.00   00:00:00.00     24    60     0     0    0     0
16.19.13 JOB00874 IEF404I DELETE - ENDED - 0 WARNING(S) - TIME=16.19.13
```

```
16.19.13 JOB00874 - ============================================================================================================
16:19:13 JOB00874 IEF373I STEP/COMPRESS/STARTED AT 2026-08-07 16:19:13
16:19:13 JOB00874 IEF574I STEP ENVAR DEBUG=TRUE
16:19:13 JOB00874 IEF574I ENVAR SYSUT1=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB
16:19:13 JOB00874 IEC040I ALLOCATING OLD/SHARED DATASET (HERC01.TEST.LOADLIB)
16:19:13 JOB00874 IEF574I ENVAR SYSUT2=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB2
16:19:13 JOB00874 IEC041I ALLOCATING NEW PDS LIBRARY (HERC01.TEST.LOADLIB2)
16:19:13 JOB00874 IEC231I USING DEFAULT TIMEOUT 20 SECONDS (empty TIME string)
------ SYSOUT BEGIN ------
DEBUG=TRUE
SYSUT1=HERC01.TEST.LOADLIB
SYSUT2=HERC01.TEST.LOADLIB2
IEB101I COPYING FROM SYSUT1 TO SYSUT2
IEB144I 1 COPIED, 1 SKIPPED, 0 REPLACED
------- SYSOUT END -------
```

14.7.2 Printout Example: User defined printout

The following example shows a printout following user definitions

```
\begin{Printout}[
 form=blue,
 paperheight=999,
 bandlines=2,
 feedlines=0,
 fitcolumns=132,
 linenumbers=true,
 punchholes=false,
 fontsize=\scriptsize
 ]
 This is LuaHBTeX, Version 1.24.0...
\end{Printout}
```

```
This is LuaHBTeX, Version 1.24.0 (TeX Live 2026)  (format=lualatex 2026.6.20)  20 AUG 2026 08:59
restricted system commands enabled.
**mdlayout.tex
(./mdlayout.tex
LaTeX2e <2025-11-01>
L3 programming layer <2026-01-19>
Lua module: luaotfload 2024-12-03 v3.29 Lua based OpenType font support
Lua module: lualibs 2023-07-13 v2.76 ConTeXt Lua standard libraries.
Lua module: lualibs-extended 2023-07-13 v2.76 ConTeXt Lua libraries -- extended
collection.
luaotfload | conf : Root cache directory is "/Users/deppe/Library/texlive/2026/t
exmf-var/luatex-cache/generic/names".
luaotfload | init : Loading fontloader "fontloader-2023-12-28.lua" from kpse-res
olved path "/usr/local/texlive/2026/texmf-dist/tex/luatex/luaotfload/fontloader-
2023-12-28.lua".
Lua-only attribute luaotfload@noligature = 1
luaotfload | init : Context OpenType loader version 3.134
Inserting `luaotfload.node_processor' in `pre_linebreak_filter'.
Inserting `luaotfload.node_processor' in `hpack_filter'.
Inserting `luaotfload.glyph_stream' in `glyph_stream_provider'.
Inserting `luaotfload.define_font' in `define_font'.
Lua-only attribute luaotfload_color_attribute = 2
luaotfload | conf : Root cache directory is "/Users/deppe/Library/texlive/2026/t
exmf-var/luatex-cache/generic/names".
Inserting `luaotfload.harf.strip_prefix' in `find_opentype_file'.
Inserting `luaotfload.harf.strip_prefix' in `find_truetype_file'.
Removing  `luaotfload.glyph_stream' from `glyph_stream_provider'.
Inserting `luaotfload.harf.glyphstream' in `glyph_stream_provider'.
Inserting `luaotfload.harf.finalize_vlist' in `post_linebreak_filter'.
Inserting `luaotfload.harf.finalize_hlist' in `hpack_filter'.
```

```
Inserting `luaotfload.cleanup_files' in `wrapup_run'.
Inserting `luaotfload.harf.finalize_unicode' in `finish_pdffile'.
Inserting `luaotfload.glyphinfo' in `glyph_info'.
Lua-only attribute luaotfload.letterspace_done = 3
Inserting `luaotfload.aux.set_sscale_dimens' in `luaotfload.patch_font'.
Inserting `luaotfload.aux.set_font_index' in `luaotfload.patch_font'.
Inserting `luaotfload.aux.patch_cambria_domh' in `luaotfload.patch_font'.
Inserting `luaotfload.aux.fixup_fontdata' in `luaotfload.patch_font_unsafe'.
Inserting `luaotfload.aux.set_capheight' in `luaotfload.patch_font'.
Inserting `luaotfload.aux.set_xheight' in `luaotfload.patch_font'.
Inserting `luaotfload.rewrite_fontname' in `luaotfload.patch_font'.
Inserting `tracingstacklevels' in `input_level_string'. (/usr/local/texlive/202
6/texmf-dist/tex/latex/koma-script/scrbook.cls
Document Class: scrbook 2026/02/02 v3.49.2 KOMA-Script document class (book)
(/usr/local/texlive/2026/texmf-dist/tex/latex/koma-script/scrkbase.sty
Package: scrkbase 2026/02/02 v3.49.2 KOMA-Script package (KOMA-Script-dependent
basics and keyval usage)
```

14.7.3 Printout Example: Plain Text

The following example shows a printout following user definitions

```
\begin{Printout}[form=plain]
 ...
\end{Printout}
```

```
16:19:13 HOST09 [INFO] $HASP373 HERC01ZZ MSGCLASS=P MSGLEVEL=(1,1) Compress a pds
16:19:13 HOST09 [INFO] IEF373I STEP/DELETE/STARTED AT 2026-08-07 16:19:13
16:19:13 HOST09 [INFO] IEF574I ENVAR LIB2DEL=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB2
16:19:13 HOST09 [INFO] IEC043I DELETING PDSE LIBRARY (HERC01.TEST.LOADLIB2)
16:19:13 HOST09 [INFO] IEC032I DELETE HERC01.TEST.LOADLIB2 (PDSE DATASET DELETED)
16:19:13 HOST09 [INFO] IEC231I USING DEFAULT TIMEOUT 20 SECONDS (empty TIME string)
16:19:13 HOST09 [INFO] IEF374I STEP/DELETE/ENDED AT 2026-08-07 16:19:13 RC=0000
16:19:13 HOST09 [INFO] IEF373I STEP/COMPRESS/STARTED AT 2026-08-07 16:19:13
16:19:13 HOST09 [INFO] IEF574I STEP ENVAR DEBUG=TRUE
16:19:13 HOST09 [INFO] IEF574I ENVAR SYSUT1=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB
16:19:13 HOST09 [INFO] IEC040I ALLOCATING OLD/SHARED DATASET (HERC01.TEST.LOADLIB)
16:19:13 HOST09 [INFO] IEF574I ENVAR SYSUT2=/data/eval/tso/JES2CAT/HERC01.TEST.LOADLIB2
16:19:13 HOST09 [INFO] IEC041I ALLOCATING NEW PDS LIBRARY (HERC01.TEST.LOADLIB2)
16:19:13 HOST09 [INFO] IEC231I USING DEFAULT TIMEOUT 20 SECONDS (empty TIME string)
16:19:13 HOST09 [INFO] IEF374I STEP/COMPRESS/ENDED AT 2026-08-07 16:19:13 RC=0000
16:19:13 HOST09 [INFO] IEF142I JOB HERC01ZZ ENDED - 0 TOTAL WARNINGS(S) - RC=0000 - 2026-08-07 16:19:13
16:19:14 HOST09 [INFO] PMC000I WTC is used for logging on HOST09.
16:19:14 HOST09 [INFO] PMC002I Print Message Class (PMC) Version 2.9.0 initiated for JOB00874.
16:19:14 HOST09 [INFO] PMC020I Line printer name: IBM9142
16:19:14 HOST09 [INFO] PMC035I Log server: logsrv05:9055
```

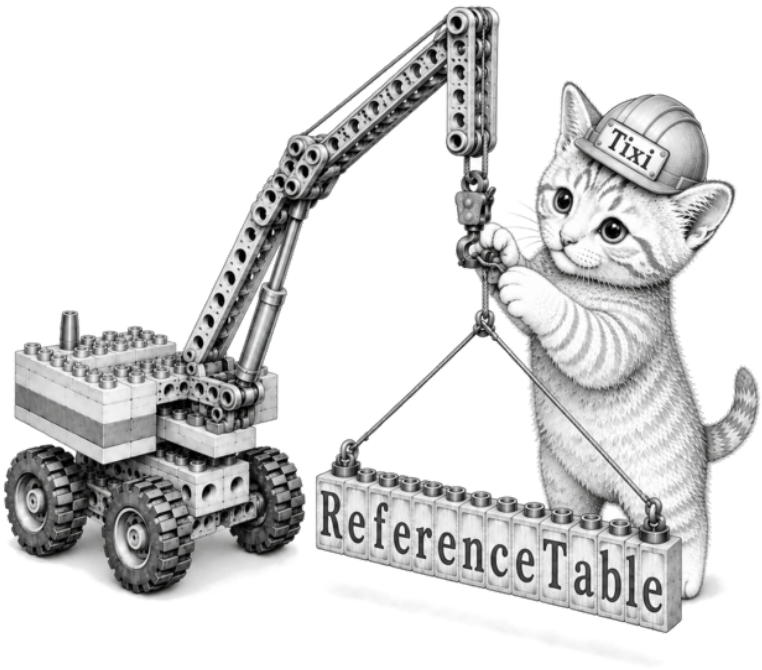
14.8 Summary

The purpose of Printout is not to provide another listing environment. Instead, it provides a semantic description of machine-generated output. The author specifies the logical properties of the printout—such as its width, paper geometry and appearance—while the package determines the appropriate typography automatically.

This approach preserves the structure of historical and contemporary computer output without sacrificing typographical quality.

Chapter 15

Reference Tables



15.1 Overview – `ReferenceTable` Environment

The `ReferenceTable` environment provides a compact and semantic interface for reference tables. It is intended primarily for tables such as command summaries, parameter descriptions, option lists, symbol tables, and similar reference material (e.g. Table 10.1, page 35 of this manual).

Internally, `ReferenceTable` handles the more verbose setup normally required for `xltabular`, including table width, repeated table headings, rules, continuation information, and the final table footer.

For example, instead of writing a complete `xltabular` definition such as

```
\begin{widelongtable}
 \begin{xltabular}{\fullwidth}{>{\ttfamily}lX}
  \caption{Command line commands} \\
  \label{tab:CommandLineCommands} \\

  \toprule
```

```
   \multicolumn{1}{l}{\textbf{Command}} &
   \textbf{Function} \\
   \midrule
   \endfirsthead

   \toprule
   \multicolumn{1}{l}{\textbf{Command}} &
   \textbf{Function} \\
   \midrule
   \endhead

   \midrule
   \multicolumn{2}{r}{\small\textit{continued...}} \\
   \bottomrule
   \endfoot

   \bottomrule
   \endlastfoot

   NEW & Creates new document ...
  \end{xltabular}
 \end{widelongtable}
```

the same table can be expressed much more compactly as

```
 \begin{ReferenceTable}[
  area=half,
  caption={ISPF Editor Command Line Commands},
  label={tab:CommandLineCommands},
  columns=2,
  verbatimcolumn=1,
  xcolumn=2
  ]{Command}{Function}

  NEW & Creates new document ...

 \end{ReferenceTable}
```

The `ReferenceTable` environment is intended primarily for tables containing commands, parameters, file names, program output, shell expressions, JCL statements, configuration values, and other material where characters having a special meaning to LaTeX must be reproduced literally.

`ReferenceTable` is therefore not merely a shorthand interface for `xltabular` or the table-area environments provided by `mdlayout`. Its `verbatimcolumn` facility changes how the contents of selected table cells are read. Characters which would normally be interpreted by TeX are accepted as literal input.

For example, a verbatim column can contain input such as

```
#COMMENT
_A_B
${HOME}
C:\TEMP\${USER}
\&SYSUID
VALUE={ABC}
~/data/a_b#c%20
\path\to\file
```

without having to protect the individual special characters for LaTeX. This is particularly useful when reference tables are generated automatically. Text intended for a verbatim column can be transferred directly into the generated LaTeX source without first applying the usual LaTeX escaping rules to characters such as

```
#  $  %  _  {  }  \  ~
```

Thus, for selected columns, `ReferenceTable` can serve as an interface between externally generated textual data and a typeset table while preserving the original textual representation.

At the same time, columns which are not declared as verbatim columns remain ordinary LaTeX columns. A single table can therefore combine literal computer text with fully processed LaTeX content. In addition to this verbatim functionality, `ReferenceTable` hides the technical setup normally required for multipage `xltabular` tables.

15.2 Basic Syntax

The number of table columns is specified with the `columns` option:

```
columns=2
columns=3
columns=8
columns=16
```

The default is

```
columns=3
```

`ReferenceTable` is not restricted to a fixed maximum of two, three, or four columns. The n-column interface can be used for considerably wider table structures where appropriate. Column numbers used by the column-property options refer to the physical columns from left to right, beginning with column 1. The traditional syntax

```
\begin{ReferenceTable}[options]
 {heading 1}{heading 2}...
 table contents
\end{ReferenceTable}
```

continues to be supported. It corresponds to

```
headers=arguments
```

and is the default header mode. For example, a traditional two-column table is written as

```
\begin{ReferenceTable}[
 columns=2
 ]{Command}{Function}
 ...
\end{ReferenceTable}
```

and the traditional three-column form does not require an explicit `columns` option:

```
\begin{ReferenceTable}{Parameter}{Command}{Meaning}
 ...
\end{ReferenceTable}
```

15.3 Table Area

The `area` option selects the horizontal area in which the table is placed.

```
area=text
area=half
area=full
```

The default is

```
area=full
```

The areas correspond to the respective areas provided by `mdlayout`. For example,

```
\begin{ReferenceTable}[
 area=text,
 columns=2
 ]{Command}{Function}
 ...
\end{ReferenceTable}
```

places the table in the normal TextArea, whereas

```
area=full
```

allows a wide reference table to use the complete FullArea.

15.4 Headers

`ReferenceTable` provides three header modes:

```
headers=arguments
headers=row
headers=none
```

The value

```
headers=false
```

is an alias for

```
headers=none
```

and may be used when the Boolean spelling appears more natural. The compatibility option

```
header=true
header=false
```

is also retained. It maps to the corresponding automatic-header and no-header modes.

15.4.1 headers=arguments

The default mode is

```
headers=arguments
```

and corresponds to the traditional `ReferenceTable` syntax:

```
\begin{ReferenceTable}[
 columns=2,
 headers=arguments
 ]{Command}{Function}

 NEW  & Creates a new document. \\
 SAVE & Saves the document. \\

\end{ReferenceTable}
```

For compatibility, the explicit `headers=arguments` declaration is normally omitted:

```
\begin{ReferenceTable}[
 columns=2
 ]{Command}{Function}
 ...
\end{ReferenceTable}
```

The heading arguments are used to construct the automatically repeated table header.

15.4.2 headers=row

For n-column tables, the headings can be supplied as the first table row:

```
headers=row
```

For example,

```
\begin{ReferenceTable}[
 area=full,
 columns=8,
 headers=row,
 xcolumn=2+4+6+8,
 verbatimcolumn=2+4+6+8
 ]

 Symb & Command & Symb & Command &
 Symb & Command & Symb & Command \\

 $\leq$    & \leq    &
 $\geq$    & \geq    &
 $\neq$    & \neq    &
 $\approx$ & \approx \\

 $\times$  & \times  &
 $\div$    & \div    &
 $\pm$     & \pm     &
 $\cdot$   & \cdot   \\

\end{ReferenceTable}
```

This syntax has the advantage that the heading has exactly the same column-oriented form as an ordinary table row. It is therefore particularly convenient for tables with many columns and for automatically generated tables whose first data record already contains the column names. The header row is separated from the following data rows and is used as the table header when the table continues on another page.

15.4.3 headers=none

Automatic headings and the corresponding automatic table rules can be disabled with

```
headers=none
```

or equivalently

```
headers=false
```

For example,

```
\begin{ReferenceTable}[
 area=text,
 columns=2,
 headers=none,
 verbatimcolumn=1,
 xcolumn=none
 ]

 TEST & First entry \\
 ABC  & Second entry \\ \hline
 XYZ  & Third entry \\

\end{ReferenceTable}
```

In this mode, `ReferenceTable` does not generate column headings or automatic table rules. Explicit table commands may still be supplied by the user. Thus commands such as

```
\hline
\toprule
\midrule
\bottomrule
\addlinespace
```

may be used where required. This mode is useful when `ReferenceTable` is wanted for its area, column-width, multipage, or verbatim capabilities, while the visual table structure is to remain completely under user control.

15.5 Caption and Label

A table caption can be specified with

```
caption={Command line commands}
```

and a label with

```
label={tab:CommandLineCommands}
```

For example,

```
\begin{ReferenceTable}[
 area=half,
 caption={ISPF Editor Command Line Commands},
 label={tab:CommandLineCommands},
 columns=2
 ]{Command}{Function}
 ...
\end{ReferenceTable}
```

The table can subsequently be referenced in the normal way:

```
Table~\ref{tab:CommandLineCommands}
```

A table carrying a caption is treated as a somewhat more distinct typographical block and therefore receives a slightly increased separation from the preceding text.

15.6 Typewriter Columns

A column can be typeset in typewriter font with

```
typewritercolumn=1
```

Multiple columns can again be specified with +:

```
typewritercolumn=1+2
```

The equivalent plural alias is

```
typewritercolumns=1+2
```

For example,

```
\begin{ReferenceTable}[
 columns=3,
 typewritercolumn=1+2,
 xcolumn=3
 ]{Parameter}{Command}{Meaning}
 ...
\end{ReferenceTable}
```

15.7 X Columns

Columns whose width should automatically adapt to the available table width can be declared as `X` columns. The preferred singular option is

```
xcolumn=2
```

Several columns are separated by +:

```
xcolumn=2+4+6+8
```

The plural spelling

```
xcolumns=2+4+6+8
```

is an equivalent alias and is retained for compatibility. For example,

```
\begin{ReferenceTable}[
 columns=2,
 xcolumn=2
 ]{Command}{Function}
 ...
\end{ReferenceTable}
```

uses a natural-width first column and an automatically expanding second column. Several X columns share the width available after the natural-width columns have been taken into account. For example,

```
xcolumn=2+4+6+8
```

makes columns 2, 4, 6, and 8 flexible. Automatic X-column sizing can explicitly be disabled with

```
xcolumn=none
```

In that case all columns use their natural width unless another column property affects their layout.

15.8 X-Column Weights

If several X columns are used, their relative widths can be controlled with the `xweights` option. The weights correspond to the X columns in the order in which they are specified by `xcolumn` or `xcolumns`.

For example,

```
xcolumns=2+3,
xweights=1:2
```

declares columns 2 and 3 as X columns and assigns them the relative widths 1:2. Consequently, column 3 is twice as wide as column 2.

A typical three-column table can therefore be defined as follows:

```
\begin{ReferenceTable}[
 caption={Example with weighted X columns},
 label={tab:WeightedXColumns},
 area=text,
 columns=3,
 typewritercolumn=1,
 xcolumns=2+3,
 xweights=1:2,
 headers=row
 ]
 Command & Meaning & Explanation \\
 foo     & Short description
 & A longer explanation which is given more
 horizontal space than the preceding column. \\
```

```
  bar     & Another description
  & Another longer explanation demonstrating
  the weighted X-column layout. \\
\end{ReferenceTable}
```

Table 15.1: Example with weighted X columns

Command	Meaning	Explanation
foo	Short description	A longer explanation which is given more horizontal space than the preceding column.
bar	Another description	Another longer explanation demonstrating the weighted X-column layout.

In this example, column 1 uses its natural width and is typeset as a typewriter column. Columns 2 and 3 share the remaining available width. Because their weights are 1:2, column 3 is twice as wide as column 2.

If `xweights` is omitted, all X columns have equal weight. Thus,

```
xcolumns=2+3
```

is equivalent to

```
xcolumns=2+3,
xweights=1:1
```

The number of weights must match the number of X columns. For example,

```
xcolumns=2+3+4,
xweights=1:2:3
```

assigns the relative widths 1:2:3 to columns 2, 3, and 4, respectively.

The weights specify *relative proportions* rather than absolute widths. They are normalized internally so that the total width available to the X columns remains unchanged. Thus, `xweights` affects only the distribution of this width among the X columns, not the overall width of the table.

15.9 Verbatim Columns

A verbatim column is fundamentally different from a typewriter column. The option

```
typewritercolumn=1
```

changes the font used for a normal LaTeX column. Its contents are still read and interpreted by TeX in the normal way. In contrast,

```
verbatimcolumn=1
```

changes how the cell contents themselves are read. Special characters are accepted literally instead of being interpreted as TEX syntax. Several verbatim columns may be selected:

```
verbatimcolumn=2+4+6+8
```

The plural form

```
verbatimcolumns=2+4+6+8
```

is an equivalent alias. For example, the following contents can be written directly into a verbatim column:

```
#COMMENT
_A#B
${HOME}
C:\TEMP
\path\to\file
\&SYSUID
VALUE={ABC}
~/data/a_b#c%20
```

No additional escaping of these characters is required for the contents of the verbatim column. This property is especially important for automatically generated tables. Suppose an external program produces the strings

```
${HOME}/data
C:\TEMP\${USER}
VALUE={ABC}
#COMMENT
```

If the target column is a `verbatimcolumn`, these strings can be written directly to the corresponding cells of the generated `ReferenceTable`. The generator does not have to translate the contents into equivalent LATEX source code first. For example, generated source may contain

```
\begin{ReferenceTable}[
 area=text,
 columns=2,
 verbatimcolumn=1,
 xcolumn=2
 ]{Value}{Description}

 ${HOME}/data    & Home data directory \\
 C:\TEMP\${USER} & Temporary directory \\
 VALUE={ABC}     & Assignment \\
 #COMMENT        & Comment marker \\

\end{ReferenceTable}
```

Value	Description
${HOME}/data	Home data directory
C:\TEMP\${USER}	Temporary directory
VALUE={ABC}	Assignment
#COMMENT	Comment marker

The first column is transferred as literal textual data, whereas the second column remains normal LaTeX and may contain formatting commands, mathematics, references, and other LaTeX constructs.

15.10 Leading Spaces in Verbatim Columns

Real verbatim processing also means that whitespace in the source can become part of the table contents. This is important because LaTeX source is normally indented for readability. By default,

```
trimleadingspaces=true
```

removes whitespace preceding the first non-space character of a verbatim cell. For example, source such as

```
echo "Hello      World"
```

is read as

```
echo "Hello      World"
```

Only whitespace before the first character is removed. Whitespace occurring after the first non-space character is preserved exactly. Thus the spaces between `Hello` and `World` in the example above remain unchanged. This behaviour makes it possible to indent generated or manually written table source without accidentally introducing indentation into the displayed verbatim cell. Leading whitespace can deliberately be preserved with

```
trimleadingspaces=false
```

This is useful when indentation at the beginning of the literal text is itself significant.

15.11 Verbatim versus Typewriter Columns

The distinction between `verbatimcolumn` and `typewritercolumn` is intentional. A `verbatimcolumn` is intended for material that must be reproduced exactly as it appears in the source. Its contents are treated as literal text rather than as ordinary LaTeX input. This is useful for command names, file names, shell syntax, source fragments, and other material where special characters and control

sequences must remain unchanged. A `typewritercolumn`, in contrast, is an ordinary LaTeX column that is merely typeset in a typewriter font. Its contents are still interpreted by LaTeX. This makes it suitable for values that are obtained from package definitions or other commands while retaining the visual appearance of technical or code-like material.

Table 10.1 on page 35 in this documentation illustrates the difference directly. Its first column is declared as

```
verbatimcolumn=1
```

Because the package option names should be reproduced exactly as written. The second column is declared as

```
typewritercolumn=2
```

because the displayed default values are obtained from public **md**layout definitions. The third column retains its default behavior as an ordinary LaTeX text column. For example, the source of Table 10.1 contains

```
marginwidth & \mdDefaultMarginwidth &
Width of the usable MarginArea. \\
```

Here, `\mdDefaultRightmargin`[1] is evaluated by LaTeX and therefore produces the default value `20mm`. The same principle is used for defaults that are expressed relative to `\baselineskip`:

```
chapterrulegap &
\mdDefaultChapterrulegapFactor\verb|\baselineskip| &
...
```

In this case, the factor is obtained from the corresponding **md**layout definition, while `\baselineskip` is deliberately shown as literal LaTeX syntax. Thus, the two column types differ not primarily in appearance, but in how their contents are interpreted:

- `verbatimcolumn`: exact reproduction of source material,
- `typewritercolumn`: normal LaTeX evaluation with typewriter-style presentation.

The complete example can be found in the source of this documentation, `mdlayout.tex`, in the definition of Table 10.1.

15.12 Column Alignment

The horizontal alignment of individual columns can be controlled independently of their width and content type. The available options are

[1] **md**layout provides public definitions for its package-option defaults. For example, `\mdDefaultRightmargin` contains the default value of the `rightmargin` option. These definitions are particularly useful when documenting **md**layout itself, since the documented values are obtained directly from the package rather than duplicated in the documentation source.

```
leftcolumn=...
centercolumn=...
rightcolumn=...
```

Several columns are specified with +:

```
leftcolumn=1
centercolumn=2+3
rightcolumn=4
```

The plural forms are equivalent aliases:

```
leftcolumns=...
centercolumns=...
rightcolumns=...
```

Left alignment is the default. Consequently, an explicit `leftcolumn` declaration is normally not required. For example,

```
\begin{ReferenceTable}[
 area=text,
 columns=3,
 leftcolumn=1,
 centercolumn=2,
 rightcolumn=3
 ]{A}{B}{C}

 left & center & right \\

\end{ReferenceTable}
```

A	B	C
left	center	right

Alignment and column width are independent properties. An `X` column can therefore also be left-, center-, or right-aligned:

```
\begin{ReferenceTable}[
 xcolumn=1+2+3,
 leftcolumn=1,
 centercolumn=2,
 rightcolumn=3
 ]{A}{B}{C}
 ...
\end{ReferenceTable}
```

A column must not be assigned more than one alignment. For example,

```
centercolumn=2,
rightcolumn=2
```

is an error.

15.13 Independent Column Properties

The column options describe independent properties and may therefore be combined freely. A column can, for example, simultaneously be

- an X column,
- a verbatim column,
- a typewriter column, and
- explicitly aligned.

For example,

```
\begin{ReferenceTable}[
 area=text,
 columns=3,
 verbatimcolumn=2+3,
 typewritercolumn=2,
 xcolumn=3,
 leftcolumn=1,
 centercolumn=2,
 rightcolumn=3
 ]{TEST A}{B}{C}

 \textbf{Normal} & #B      & ${HOME} \\
 $x^2$           & C:\TEMP & \path\to\file \\

\end{ReferenceTable}
```

TEST A	**B**	**C**
Normal	#B	${HOME}
x^2	C:\TEMP	\path\to\file

In this example the columns have the following properties:

Column	Width	Contents	Alignment
1	natural	normal LaTeX	left
2	natural	verbatim/typewriter	centered
3	X	verbatim	right

Thus width, font/content processing, and alignment are independent characteristics of a column. The same principle applies to n-column tables. For example,

```
xcolumn=2+4+6+8,
verbatimcolumn=2+4+6+8
```

makes the four even-numbered columns both flexible X columns and literal verbatim columns.

15.14 Additional Space and Commands Between Rows

Commands which modify the space between table rows may be placed after the row separator. For example,

```
NEW & Creates new document \\ \addlinespace
CLEAR & Clear document \\
```

An explicit amount can also be specified:

```
A & First row  \\ \addlinespace[5ex]
B & Second row \\
```

Several table commands can follow the row separator. For example,

```
DATASET, DATS &
Toggles the currently selected dataset on/off. \\
\addlinespace[5ex] \hline \addlinespace

NEXT & Next entry \\
```

The commands following \\ are applied between the completed row and the following data row. This post-row mechanism is also available when one or more cells of the row are verbatim cells.

15.15 N-Column Example: Math-Mode Symbols

The following example demonstrates an eight-column table consisting of four pairs of symbol and command columns. The command columns are simultaneously X columns and verbatim columns.

Table 15.5: LaTeX Math Symbols

Symb	Command	Symb	Command	Symb	Command	Symb	Command
≤	\leq	≥	\geq	≠	\neq	≈	\approx
×	\times	÷	\div	±	\pm	·	\cdot
˚	^{\circ}	∘	\circ	′	\prime	⋯	\cdots
∞	\infty	¬	\neg	∧	\wedge	∨	\vee
⊃	\supset	∀	\forall	∈	\in	→	\rightarrow
⊂	\subset	∃	\exists	∉	\notin	⇒	\Rightarrow
∪	\cup	∩	\cap	∣	\mid	⇔	\Leftrightarrow
$\dot{a}$	\dot a	$\hat{a}$	\hat a	$\bar{a}$	\bar a	$\tilde{a}$	\tilde a
α	\alpha	β	\beta	γ	\gamma	δ	\delta
ϵ	\epsilon	ζ	\zeta	η	\eta	ε	\varepsilon
θ	\theta	ι	\iota	κ	\kappa	ϑ	\vartheta
λ	\lambda	μ	\mu	ν	\nu	ξ	\xi
π	\pi	ρ	\rho	σ	\sigma	τ	\tau

continued…

Symb	Command	Symb	Command	Symb	Command	Symb	Command
υ	\upsilon	ϕ	\phi	χ	\chi	ψ	\psi
ω	\omega	Γ	\Gamma	Δ	\Delta	Θ	\Theta
Λ	\Lambda	Ξ	\Xi	Π	\Pi	Σ	\Sigma
Υ	\Upsilon	Φ	\Phi	Ψ	\Psi	Ω	\Omega

Table 15.5 was typeset by:

```
\begin{ReferenceTable}[
  caption={\LaTeX{} Math Symbols},
  label={tab:LaTeXmathSymbols},
  area=full,
  columns=8,
  headers=row,
  xcolumn=2+4+6+8,
  verbatimcolumn=2+4+6+8
  ]

  Symb & Command & Symb & Command &
  Symb & Command & Symb & Command \\

  $\leq$       & \leq       &
  $\geq$       & \geq       &
  $\neq$       & \neq       &
  $\approx$    & \approx    \\
  ...
  $\Omega$     & \Omega     \\

\end{ReferenceTable}
```

The example also illustrates an important property of the n-column engine: the same physical column may simultaneously be declared as an X column and as a verbatim column.

15.16 Complete Command-Table Example

The following example shows a command reference table as it could occur in software documentation. Its contents are entirely fictitious and serve only to demonstrate the structure and formatting capabilities of the `ReferenceTable` environment; they do not document an actual command set.

Important:
`verbatimcolumn` does not mean “use a typewriter font”. It means “treat the contents of this column as literal text”. The typewriter appearance is a consequence of the verbatim column.

```
\begin{ReferenceTable}[
  area=half,
  caption={ISPF Editor Command Line Commands},
  label={tab:CommandLineCommands},
  columns=2,
  verbatimcolumn=1,
  xcolumn=2
  ]{Command}{Function}

  NEW &
  Creates new document
    \texttt{NEW-\textless{}creation time\textgreater{}.TXT}
    \\ \addlinespace

  CLEAR & Clear document \\
  SAVE  & Save document \\
  END   & Save file and return \\
  FIND, F <t1> <t2> ... &
    Marks occurrence(s) of \texttt{<t1> <t2>...} \\
  FIND NEXT, F N &
    Finds next occurrence of \texttt{<t1> <t2>...} \\
  RFIND, F5 & Repeats \texttt{FIND NEXT} \\
  RESET, RES & Deletes color highlighting \\
  COLS, COLUMNS & Toggles the column ruler on/off \\
  PATH & Toggles path info on/off \\
  CANCEL, QUIT, EXIT & Exit from EDIT and return \\
\end{ReferenceTable}
```

Table 15.6 shows the table defined above, with a column in verbatim format (column 1) on the left. The right-hand column (column 2) contains standard text and requires LaTeX formatting—such as `\texttt{FIND NEXT}`—to achieve a typewriter-style appearance.

Table 15.6: ISPF Editor Command Line Commands

Command	Function
NEW	Creates new document NEW-<creation time>.TXT
CLEAR	Clear document
SAVE	Save document
END	Save file and return
FIND, F <t1> <t2> ...	Marks occurrence(s) of <t1> <t2>...
FIND NEXT, F N	Finds next occurrence of <t1> <t2>...
RFIND, F5	Repeats FIND NEXT
RESET, RES	Deletes color highlighting

continued...

Command	Function
`COLS, COLUMNS`	Toggles the column ruler on/off
`PATH`	Toggles path info on/off
`CANCEL, QUIT, EXIT`	Exit from EDIT and return

15.17 Option Summary

Table 15.7: Option Summary, Table is typeset as *ReferenceTable*

Option	Default	Meaning
`area`	full	Table area
`caption`	—	Table caption
`label`	—	Table label
`columns`	3	Number of columns
`headers`	arguments	Header mode: arguments, row, none/false
`header`	true	Compatibility header switch
`xcolumn(s)`	last column	Automatically sized columns
`typewritercolumn(s)`	—	Typewriter columns
`verbatimcolumn(s)`	—	Verbatim columns
`leftcolumn(s)`	all	Left-aligned columns
`centercolumn(s)`	—	Centered columns
`rightcolumn(s)`	—	Right-aligned columns
`trimleadingspaces`	true	Trim leading whitespace in verbatim cells

Table 15.7 was typeset using the first row of data as column headers via the `headers=row` option:

```
\begin{ReferenceTable}[
 caption={Option Summary, Table is typeset as \emph{ReferenceTable}},
 label={tab:OptionSummary},
 area=text,
 columns=3,
 typewritercolumn=1,
 xcolumn=3,
 headers=row
 ]
 Option              & Default   & Meaning \\
 area                & full      & Table area \\
 caption             & ---       & Table caption \\
 label               & ---       & Table label \\
```

```
 columns             & 3         & Number of columns \\
 headers             & arguments & Header mode: arguments, row, none/false \\
 header              & true      & Compatibility header switch \\
 xcolumn(s)          & last column & Automatically sized columns \\
 typewritercolumn(s) & ---       & Typewriter columns \\
 verbatimcolumn(s)   & ---       & Verbatim columns \\
 leftcolumn(s)       & all       & Left-aligned columns \\
 centercolumn(s)     & ---       & Centered columns \\
 rightcolumn(s)      & ---       & Right-aligned columns \\
 trimleadingspaces   & true      & Trim leading whitespace in verbatim cells \\
\end{ReferenceTable}
```

The singular and plural forms of the column-property options in Table 15.7 are aliases. For example,

```
 xcolumn=1+3
```

and

```
 xcolumns=1+3
```

are equivalent. Likewise,

```
 headers=false
```

is an alias for

```
 headers=none
```

15.18 Notes and Current Restrictions

`ReferenceTable` supports an arbitrary positive number of physical table columns through the `columns` option. Column numbers used with `xcolumn`, `verbatimcolumn`, `typewritercolumn`, and the alignment options refer to the physical columns from left to right, starting with column 1. The traditional

```
 {Heading 1}{Heading 2}...
```

interface is provided by `headers=arguments`. For general n-column tables, `headers=row` provides the more natural interface because the first row has exactly the same column structure as the table body. If no automatically generated heading is required, use

```
 headers=none
```

or its alias

```
 headers=false
```

In that mode the user may provide table rules explicitly. The `ReferenceTable` interface deliberately hides the underlying `xltabular` / `longtable` implementation. Code using `ReferenceTable` should therefore describe the semantic properties of the table through the provided options instead of relying on the internal column specification.

This separation is intentional: the purpose of `ReferenceTable` is not merely to shorten an `xltabular` declaration, but to provide a stable semantic table interface in which column width, alignment, font, literal/verbatim processing, table area, and header generation can be specified independently.

Chapter 16

Generating HTML and DOCX from mdlayout

The same LaTeX source used to produce a PDF can also be converted into a semantic HTML document. The HTML version does not attempt to reproduce a paginated PDF pixel by pixel. Instead, it translates the structural concepts of **md**layout into a responsive web layout while retaining the narrow reading column, the additional layout areas, cross-references, mathematics, tables, figures, and code-like material.

The DOCX output is primarily intended to make individual text passages available for further editing or reuse in word processors such as OpenOffice, LibreOffice, Apple Pages or, if necessary, Microsoft Word. It should be emphasized that the DOCX output is *not* intended to be a "Word version" of the LaTeX-generated PDF. LaTeX and conventional word processors follow fundamentally different approaches to document preparation and typesetting. In particular, complex page layouts, precise typography, automatic placement and alignment, cross-references, custom environments, and other LaTeX-specific constructs can only be reproduced to a *limited* extent in a word-processor format[1]. Consequently, the DOCX output should be regarded as a convenient interchange and editing format rather than as an alternative publication format.

> ☞ The authoritative representation of the document remains the LaTeX (`*.pdf`) version, which is designed to meet the highest typographical and layout standards. No attempt is made to reproduce its appearance faithfully in DOCX.

16.1 Requirements

The supplied HTML/DOCX tool set requires:

- Pandoc 3 or later,
- Python 3,

[1] **Why DOCX Lacks Reliability:** Programs like LibreOffice, Google Docs, and Microsoft Word interpret the complex Office Open XML standard differently, which shifts layouts and margins. If a document uses a font your computer does not have, the software substitutes another font, altering line breaks and page counts. DOCX is designed to adjust text dynamically based on screen size and window dimensions rather than locking elements in place like a printed page. The official DOCX standard is thousands of pages long, making full, bug-free implementation difficult. Thus, even for Microsoft Word, the only truly *reproducible* final format is PDF.

- a modern web browser, and
- optionally, Poppler and ImageMagick for converting directly included PDF graphics into browser-friendly images.

If you want to generate the used typewriter font from the freely available[2] TrueType font `IBMPlexMono-Regular.ttf` by

```
python3 html-build/make-mono-webfont.py \
 IBMPlexMono-Regular.ttf \
 html-build/fonts/mdlayout-plex-mono-85.ttf
```

you need the Python libraries `pathlib` and `fontTools`. These can be installed by

```
pip3 install --upgrade pip
pip3 install fonttools pathlib
```

All commands used here should be entered via a Unix/Linux shell, with the `mdlayout` base directory as the present working directory (`pwd`).

16.1.1 macOS with Homebrew

On macOS, the required tools can be installed with Homebrew:

```
brew install pandoc python
```

Poppler and ImageMagick are optional:

```
brew install poppler imagemagick
```

The `poppler` formula supplies both `pdftocairo` for preferred vector conversion and `pdftoppm` for the high-resolution raster fallback. These programs are not needed when a matching SVG or WebP image is already supplied in `html-build/images`. However, if the supplied script `extract-images.sh` is to be used for converting images, `poppler` and `imagemagick` must be installed.

16.1.2 Debian and Ubuntu Linux

On Debian, Ubuntu, and related Linux distributions using APT, install the corresponding packages with:

```
sudo apt update
sudo apt install pandoc python3
```

The optional graphics converters can be installed separately:

```
sudo apt install poppler-utils imagemagick
```

The package `poppler-utils` provides `pdftocairo` and `pdftoppm`. Package versions depend on the selected distribution release. If the repository supplies a Pandoc version older than 3, install a current release from pandoc.org.

[2] https://github.com/IBM/plex/blob/master/packages/plex-mono/fonts/complete/ttf/IBMPlexMono-Regular.ttf

16.1.3 Verifying the Installation

The essential programs can be checked before the first build:

```
pandoc --version
python3 --version
```

If the optional graphics converters have been installed, they can additionally be checked with:

```
pdftocairo -v
pdftoppm -v
magick -version
```

Depending on the installed ImageMagick version, the final command may instead be:

```
convert -version
```

A command-not-found response for `pdftocairo`, `pdftoppm`, or ImageMagick does not prevent the HTML build when all PDF figures already have matching browser images. The tools are required only when the converter must create a new SVG or WebP image from a PDF.

Place the `html-build` directory next to the main LaTeX source file. The converter automatically reads `mdlayout.sty` and `mdpreamble.tex` from the source directory when they are available.

16.2 Building the HTML / DOCX Document

The normal build command is:

```
make -f html-build/Makefile html
```

for the HTML version and:

```
make -f html-build/Makefile docx
```

for the word processor version. Both versions can also be generated at once:

```
make -f html-build/Makefile html docx
```

The conversion script can also be called directly:

```
html-build/build-html.sh mdlayout.tex mdlayout.html
```

A differently named style file and an explicit preamble may be supplied as the third and fourth arguments:

```
html-build/build-html.sh \
 mdlayout.tex mdlayout.html \
 mdlayout.sty mdpreamble.tex
```

The generated `mdlayout.html` contains its stylesheet and the scaled IBM Plex Mono webfont.

> ☞ Image files remain in the adjacent `images` directory and must be distributed together with the HTML file.

In analogy, the DOCX file can be built by specifying the input files explicitly:

```
html-build/build-docx.sh \
mdlayout.tex mdlayout.docx \
mdlayout.sty mdpreamble.tex
```

16.3 Conversion Pipeline

The conversion is deliberately divided into several stages:

1. The Python preprocessor protects verbatim material, evaluates HTML-specific conditionals, expands selected definitions, collects labels and captions, and converts special environments into stable markers.
2. Pandoc translates standard LaTeX structures into HTML.
3. A Lua filter converts **md**layout constructs such as layout areas, `ReferenceTable`, `Printout`, margin figures, and synchronized content into semantic HTML elements.
4. The postprocessor completes the document directories, resolves local graphics, and prepares the final browser document.

This staged approach is necessary because Pandoc reads LaTeX syntax but does not execute the definitions contained in a package file.

16.4 Layout Areas in HTML

On sufficiently wide browser windows, the principal **md**layout areas retain their semantic widths and positions:

`TextArea` contains ordinary prose and uses the comfortable default reading width.

`MarginArea` forms the additional column to the left of the gutter.

`HalfArea` extends the TextArea into half of the additional horizontal space.

`FullArea` uses the combined width of MarginArea, gutter, and TextArea.

`DirectoryArea` provides the intermediate width used for document directories.

Below the responsive breakpoint, these areas collapse to the available browser width. This avoids horizontal scrolling on tablets and small screens while preserving the semantic distinction in the HTML structure.

The conventional LaTeX command `\marginpar` remains independent of the **md**layout `MarginArea`. If sufficient browser width is available, the note is placed in a separate right-hand margin. Otherwise, it becomes a compact note inside the normal text flow.

16.5 Document Directories and References

The commands

```
\tableofcontents
\listoffigures
\listoftables
```

are evaluated at their positions in the source. They therefore remain inside a surrounding `DirectoryArea` and retain the order selected by the author. Entries use the document's chapter, section, figure, and table numbers and link directly to their HTML targets.

16.6 Images and Mathematics

Image widths expressed relative to `\mdTextWidth`, `\mdMarginWidth`, `\mdFullWidth`, `\linewidth`, or `\textwidth` are converted into relative CSS widths. Named pages from `images.pdf` are read according to `mdlayout-images-pages.tex` and exported as WebP images.

Mathematics is retained as mathematical markup and rendered by MathJax in the browser. Equation numbers and the special surface-integral operators used in this documentation are preserved by the conversion filters.

16.7 Conditional PDF and HTML Content

Some material is meaningful only in one output format. The public conditionals `\ifPdf` and `\ifHtml` allow both versions to be maintained in the same source. For example:

```
\ifPdf
 This paragraph is included only in the PDF.
\else
 This paragraph is included only in the HTML document.
\fi
```

The complementary form can be used when no alternative is required:

```
\ifHtml
 This additional explanation is specific to HTML.
\fi
```

In a normal LaTeX build, `\ifPdf` is true and `\ifHtml` is false. The HTML preprocessor selects the opposite values. Both forms support `\else` and may be nested. Other TeX conditionals are passed through unchanged.

> ☞ Conditional sections should be used only where the output formats genuinely require different content. Material that can be represented meaningfully in both PDF and HTML should remain in the common document source.

16.8 Limitations

The HTML converter supports the standard LaTeX structures and the **md**layout constructs documented by this manual. Arbitrary user-defined TeX macros are not executed automatically. A new semantic command or environment may therefore require a corresponding preprocessing or Lua-filter rule.

Page-dependent concepts such as physical page numbers, forced page breaks, running headers, and exact float placement do not have direct equivalents in a continuously scrolling HTML document. The converter preserves their semantic content where useful rather than imitating paper-specific behaviour.

Chapter 17

Compatibility

17.1 Multicolumn Text

The standard `multicols` environment provided by the `multicol` package can be used seamlessly within the `TextArea`. Column widths are calculated from the current `TextArea` width and therefore require no special treatment by **md**layout. For example, the following code

```
\lipsum[1][1-5]
\begin{multicols}{2}
 \lipsum[1][6-10]
\end{multicols}
\lipsum[1][11-25]
```

produces a paragraph in which only the middle part is typeset in two columns:

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat.

Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

The `multicols` environment can also be used within `FullArea`. Its columns automatically adapt to the width of the surrounding area:

```
\begin{FullArea}
 \lipsum[1][1-5]
 \begin{multicols}{3}
```

```
  \lipsum[1][6-15]
 \end{multicols}
 \lipsum[1][16-30]
\end{FullArea}
```

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla.

Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

17.2 LaTeX Margin Notes

The standard LaTeX command `\marginpar` can be used with **md**layout. It is important, however, not to confuse the LaTeX margin used for margin notes with the `MarginArea` provided by **md**layout.

The `MarginArea` is an integral part of the horizontal page layout defined by **md**layout:

```
MarginArea | Gutter | TextArea
```

It is intended for document elements that explicitly use the **md**layout area model. In contrast, `\marginpar` retains its normal LaTeX meaning. Margin notes are placed in the conventional LaTeX margin outside the text area. They are therefore not placed in the **md**layout `MarginArea`. For example,

```
This is text in the TextArea.
\marginpar{A LaTeX\\ margin\\ note.}
The paragraph continues normally.
```

produces the conventional LaTeX margin note:

This is text in the `TextArea`. The paragraph continues normally.

A LaTeX margin note.

For comparison, text can explicitly be placed in the `MarginArea` using the **md**layout area model:

```
\SyncMarginAndTextArea{This is text in the \texttt{MarginArea}.}
{This is text in the \texttt{TextArea}.}
```

which produces:

This is text in the MarginArea.

This is text in the TextArea.

The default **md**layout setting of

```
rightmargin=20mm
```

provides relatively little space for conventional margin notes. Consequently, longer notes may appear too narrow or may not produce a useful layout. If conventional \marginpar notes are an important part of a document, a larger right margin should be selected, for example:

```
\usepackage[
rightmargin=35mm
]{mdlayout}
```

This additional space is independent of the **md**layout MarginArea on the left. Thus, both concepts can coexist in the same document, but they serve fundamentally different purposes: \marginpar creates a conventional LaTeX margin note outside the TextArea, whereas the MarginArea is part of the structural page model of **md**layout and can contain ordinary document content.

> ☞ It should be emphasized that the MarginArea is not related to the LaTeX page margins. It constitutes a spatially completely separate area and does not overlap with them. In a standard LaTeX or KOMA-Script layout, this additional text area *does not exist at all*.

17.3 Standalone Use of Subpackages

The ReferenceTable and Printout environments are implemented in separate package files and can currently also be used without loading the main mdlayout package.

This permits the environments to be used in documents which do not otherwise use the mdlayout layout model. For example, a document requiring only reference tables may load

```
\usepackage{mdlayout-referencetable}
```

and a document requiring only the continuous-paper printout environment may load

```
\usepackage{mdlayout-printout}
```

Both packages determine the geometry required for their environments from standard LaTeX dimensions when the corresponding mdlayout dimensions are not available.

In standalone operation, the basic geometry is derived from

```
\textwidth
\marginparwidth
\marginparsep
```

The margin area and the separation between the margin and the normal text area therefore have to be defined appropriately by the document class or by another layout package. They may, for example, be established with `geometry`:

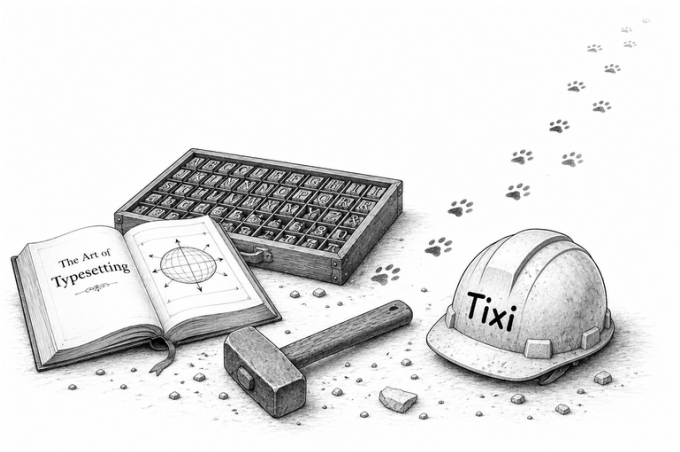
```
\usepackage[
left=50mm,
right=20mm,
marginparwidth=30mm,
marginparsep=5mm
]{geometry}

\usepackage{mdlayout-printout}
```

The same principle applies to `mdlayout-referencetable`. If `mdlayout` is present, however, both subpackages use the layout dimensions calculated by `mdlayout` and therefore remain synchronized with its text, margin, and full-width areas.

> ☞ Standalone use of `mdlayout-referencetable` and `mdlayout-printout` is provided primarily as a convenience and is *not the recommended mode of operation*.
>
> The supported and preferred configuration is to load the main `mdlayout` package and let it load the corresponding subpackages. This ensures that the subpackage versions and their layout calculations are consistent with the version of `mdlayout` in use.
>
> Standalone operation relies on the subpackages being able to derive a suitable layout from standard LaTeX dimensions. Although this is possible with the current implementation, it is not part of the guaranteed long-term interface of `mdlayout`. Future versions may change the internal requirements of these environments and may therefore restrict or remove standalone operation.

Tixi's work here is done …



Tixi has left her tools behind, and her paw prints slowly disappear beyond these pages.

*Perhaps, one day, they will lead her back to us — for another **md**layout or LaTeX project.*